基于状态空间偏差识别与事件级 MILP 的多源定位融合及机器人任务优化模型

摘要

多源定位数据不能只按轨迹残差大小评价，其误差会继续进入速度、加速度、视场边界和任务时间窗判断。本文采用“时间参数化轨迹对齐–固定偏差识别与 RTS 平滑–残差泛化检验–事件级 MILP”的求解框架，把可稳定识别的误差写入名义轨迹，把不能稳定外推的残差留给风险复核，最后在同一候选事件池中联合安排射击和拍照任务。

附件 1 的主要误差是常值时间错位。通过两源轨迹 RMSE 搜索，得到最优时间偏移 $\Delta = -198.4317$ s，对齐后 RMSE 约为 9.06×10^{-8} m，并输出 7004 个 10 Hz 融合轨迹点；该附件经时间对齐后可视为同一真实轨迹的高精度重采样。附件 2 同时存在时间错位、固定平移偏差和随机噪声，估得时间偏移 $\Delta = -50.4947$ s，方式 2 相对方式 1 的固定偏差约为 $(3.480, -1.822)$ m，校正后 RMSE 约为 0.7315 m。RTS 平滑对位置 RMSE 的改善有限，但能稳定速度和加速度，因此固定偏差进入名义轨迹，随机噪声通过状态空间平滑降低对运动学量的放大影响。

附件 3 的时间偏移为 $\Delta = 367.8843$ s。为判断实际残差是否应被强校正，本文比较 M0–M4 偏差结构模型，并采用 5 折时间块交叉验证。固定偏差模型 M1 的平均 CV-RMSE 为 2.7393 m；M3、M4 虽然训练误差更低，但平均 CV-RMSE 分别为 3.0289 m、2.8636 m，均劣于 M1。由此判定，附件 3 不采用复杂模型强校正，复杂残差结构只作为风险复核信息进入任务优化。

在任务层面，本文生成 981 个射击候选事件、47 个拍照候选事件，联合候选池共 1028 个事件；进一步构造 43744 条时间冲突边和 467 条拍照角度冲突边，并用事件级 MILP 联合优化射击与拍照选择。主方案 A 覆盖 9 个射击目标和 6 个拍照目标，射击期望命中数为 8.0998，综合效用为 14.0998；保守方案 B 牺牲部分收益，提高裕度和风险权重，将高风险事件数降为 0。A 对应收益优先，B 对应鲁棒保守，分别服务于不同执行偏好。

上述结果表明，时间错位和固定偏差可作为名义轨迹修正量，未通过时间块外推检验的残差应保留为风险约束。事件级 MILP 在收益、冲突和风险之间给出统一优化口径，使最终任务方案与前端定位误差处理保持一致。

关键词：多源定位融合；时间对齐；固定偏差识别；RTS 平滑；残差泛化检验；混合整数规划；任务调度

一、问题分析与总体建模思路

1.1 问题理解与目标拆解

题目给出两种定位方式下的机器人轨迹、18 个射击目标和 18 个拍照目标。表面上看，4 个子问题分别处理时间对齐、偏差识别、残差检验和任务调度；但从决策链条看，它们讨论的是同一个核心问题：定位误差如何沿着“轨迹估计 → 运动学量计算 → 候选事件判定 → 最终任务选择”逐层传递。若前 3 问处理失真，问题四中距离、速度、加速度和时间窗口的判断都会被系统性污染。

表 1　四个子问题的目标拆解

子问题	核心目标	主要难点	向下一问提供的输出
问题一	识别附件 1 两种定位方式的时间偏差，并输出统一时间轴上的融合轨迹	采样频率不同、时间起点不一致，不能逐行比较	对齐后的公共时间轴与 10 Hz 融合轨迹
问题二	在含噪数据中分离时间错位、固定偏差与随机噪声	时间偏差与空间偏差耦合，运动学量对噪声敏感	固定偏差估计、平滑位置以及稳定速度/加速度
问题三	判断附件 3 残差结构能否稳定外推，决定是否强校正	局部结构可能降低训练误差，却未必具有跨时间泛化性	是否保留强校正、以及事件级风险场景
问题四	在固定轨迹上联合安排射击与拍照任务	时间冲突密集，收益递减、FOV、多角度和风险约束并存	最终任务事件组合与收益–风险方案

1.2 关键难点分析

本题的难点在于前端误差会改变后端候选事件的边界判断。时间偏差若估错，固定偏差会被错误放大；固定偏差若与随机噪声混淆，速度和加速度阈值附近会出现虚假的可行窗口；若把附件 3 中的局部残差结构直接硬修到轨迹上，又会把短时波动误当成稳定规律。

因此，第一章只确定建模分工：问题一、二负责把稳定误差写入轨迹，问题三负责判断剩余残差是否具备外推价值，问题四负责在固定轨迹和风险口径下选择任务。后文各章不再重复完整主线，而分别回答“如何识别”“是否校正”和“怎样调度”。

1.3 总体求解框架

本文建立“误差分层 + 残差风险复核 + 事件级优化”的总体框架。先用时间参数化轨迹完成双源时间对齐，解决“同一物理时刻是否对上”的问题；再通过固定偏差估计和状态空间平滑，分离稳定系统偏差与随机噪声；随后用时间块交叉验证检验附件 3 残差结构的可泛化性，把外推不稳定的部分保留为风险信息；最后在固定轨迹上枚举事件级候选，并通过 MILP 同时处理收益、冲突和风险偏好。

这一框架的核心判断是：时间偏差、系统偏差和随机扰动不应被压缩成同一种修正量。能稳定识别的部分进入名义轨迹，不能稳定识别的部分进入风险约束；前者提高轨迹可信度，后者保护任务决策可靠性。

本文四个子问题之间的递进关系和总体建模流程如图 1 所示。

图 1　多源定位融合与固定轨迹任务调度总体建模框架

图 1 将四个子问题按误差处理和任务决策链条连接起来。问题一先解决两源轨迹的时间对齐，并在公共时间轴上输出 10 Hz 融合轨迹；问题二在含噪数据中识别固定偏差，再通过状态空间模型、卡尔曼滤波和 RTS 平滑得到稳定的位置、速度和加速度；问题三不直接把实际残差全部写入轨迹，而是用 M0–M4 模型和时间块交叉验证判断其是否具有稳定泛化能力；问题四把固定轨迹、运动学量和风险信息共同转化为事件级 MILP，在候选事件、时间冲突、拍照角度冲突和风险惩罚之间进行联合选择。该流程体现了本文的核心口径：稳定误差用于轨迹修正，不稳定残差转化为风险复核，最终任务表则在收益、冲突和风险之间全局权衡。

1.4　各子问题的方法对应关系

表 2 给出了各子问题之间的输入输出关系。问题一先把两源数据放到同一物理时刻下比较；问题二在此基础上估计稳定空间偏差，并输出后续会用到的速度、加速度状

表 2　子问题与方法的对应关系

子问题	核心方法	选择原因	作用
问题一	时间参数化轨迹 + RMSE 搜索	两源轨迹存在回环与异步采样，不能写成单值函数逐行比较	在统一物理时刻上完成时间对齐并输出公共轨迹
问题二	鲁棒中位数 + 卡尔曼滤波/RTS 平滑	固定偏差需要与随机测量误差分离，且后续任务依赖稳定运动学量	估计稳定系统偏差并获得平滑速度、加速度序列
问题三	M0–M4 模型族 + 时间块交叉验证	复杂残差模型可能过拟合局部时间段，必须检验跨时间泛化能力	决定残差是进入“强校正”还是进入“风险复核”
问题四	事件级候选生成 + MILP 联合优化	时间冲突、FOV、多角度、收益递减和风险惩罚都可线性表达	在固定轨迹上得到给定候选集和权重下的 MILP 最优解

态；问题三检验附件 3 残差结构是否能跨时间块保持有效；问题四只接收已经固定的轨迹口径和风险口径，再生成事件候选并求解调度模型。按这一接口传递结果时，某一阶段得到的残差改善不会直接被等同为任务机会，必须经过外推检验或风险复核后才能影响任务选择。

需要注意的是，任务规划关心的不是轨迹曲线本身有多平滑，而是候选事件是否被正确保留。远离阈值的轨迹误差通常只改变位置图形；靠近距离、速度、加速度、FOV 或准备时间边界的误差，则会改变 MILP 的候选集。因此，本文对校正量的取舍同时考察残差大小、跨时段稳定性和任务边界影响，而不是单独依据训练误差决定是否修正轨迹。

二、符号说明与基本假设

2.1 符号说明

表 3 主要符号表

符号	含义
t, Δ	原始时间与方式 2 对齐偏移量，其中 $t_2^{\text{aligned}} = t_2 + \Delta$ / s
$\boldsymbol{p}_i(t)$	第 i 种定位方式在时刻 t 的二维位置 / m
$\boldsymbol{p}_f(t)$	10 Hz 融合轨迹位置 / m
$v(t), a(t)$	由平滑轨迹计算的速度和加速度模长 / (m/s), (m/s^2)
$\boldsymbol{X}$	状态向量 $[x, y, v_x, v_y, b_x, b_y]^\top$
$\boldsymbol{b}$	方式 2 相对方式 1 的固定平移偏差 / m
Q, R_1, R_2	过程噪声协方差与两源观测噪声协方差
r_x, r_y	固定偏差校正后的 x/y 方向残差 / m
$\boldsymbol{\psi}$	问题三偏差结构模型的状态特征向量
$\mathcal{E}^S, \mathcal{E}^P$	射击候选事件集合与拍照候选事件集合
e, q_e	候选任务事件及其统一选择变量
α	拍照事件中的相机朝向角 / 度
x_e, z_e	射击事件、拍照事件选择变量
$y_{k,m}$	射击目标 k 是否获得第 m 次射击的槽位变量
u_p	拍照目标 p 是否被至少覆盖一次
C_e	拍照事件 e 覆盖的目标集合
$g_m, P_k(n_k)$	第 m 次射击边际收益与目标 k 的累计命中概率
$E_{\text{shoot}}, C_{\text{photo}}, O_{\text{photo}}$	射击期望命中、拍照目标覆盖数、拍照总观测次数
B_{multi}	多目标同拍奖励
$m_e^{\text{nom}}, m_e^{\text{worst}}, R_{\text{risk}}$	名义裕度、扰动场景最坏裕度与分段风险惩罚项
$M_1, \ldots, M_5$	MILP 目标函数权重

2.2 基本假设

采用如下基本假设：两种定位方式观测的是同一机器人；相邻采样点之间机器人运动连续；附件 1 无随机噪声，主要处理时间错位；附件 2 的主要误差可分解为固定平移偏差与随机测量误差；附件 3 是否存在稳定系统偏差需要通过时间块交叉验证判断；问题四中的机器人轨迹固定，不做路径重规划；射击和拍照共享同一时间轴，准备窗口重叠时不能同时执行；拍照时相机可在执行瞬间调整到离散候选朝向角 α，其调整时间已并入 0.5 s 准备窗口，因此问题四只优化“是否在某时刻以某一朝向执行拍照”，不再单独建模连续转台动力学。

三、问题一：无噪声双源轨迹的时间对齐

3.1 问题分析与模型选择

附件 1 给出的是无噪声双源轨迹，两条轨迹对应同一运动过程，主要差异来自时间起点不一致。建模前先统一列名、坐标单位和时间单位，并检查时间列单调性与重复采样点。由于两源采样频率分别为 4 Hz 和 5 Hz，原始数据行号不对应同一物理时刻，因此不能直接逐行相减来估计偏差。另一方面，轨迹在平面上可能出现回环或近垂直段，直接写成 $y = f(x)$ 会丢失时间顺序，也难以保持同一物理时刻上的配准关系。

基于上述特点，问题一采用时间参数化轨迹表示，并在公共重叠区间上用 RMSE 搜索时间偏差。这样处理后，双源轨迹可以在统一时间轴上比较，再进一步生成 10 Hz 融合轨迹，为后续速度、加速度和任务窗口判断提供输入。

3.2 模型构建与求解流程

对两种定位方式分别构造时间参数化插值轨迹

$$\boldsymbol{p}_i(t) = \left(S_{i,x}(t), S_{i,y}(t)\right)^\top, \quad i = 1, 2. \tag{1}$$

其中 $S_{i,x}(t)$ 和 $S_{i,y}(t)$ 是第 i 种定位方式在时间轴上的插值函数，单位为 m。这样做的前提是机器人运动在相邻采样点之间连续，且采样间隔足够小；它的优点是保留时间顺序，缺点是在剧烈加速段会受到插值光滑性的影响。给定 Δ 后，由于 $t_2^{\text{aligned}} = t_2 + \Delta$，方式 2 在公共时间轴上的取值写作 $\boldsymbol{p}_2(t-\Delta)$，其与方式 1 的差异为 $\boldsymbol{p}_2(t-\Delta) - \boldsymbol{p}_1(t)$。在重叠网格 $\mathcal{T}_\Delta$ 上定义

$$J(\Delta) = \sqrt{\frac{1}{|\mathcal{T}_\Delta|} \sum_{t \in \mathcal{T}_\Delta} \|\boldsymbol{p}_2(t-\Delta) - \boldsymbol{p}_1(t)\|_2^2}. \tag{2}$$

这里 $J(\Delta)$ 是问题一用于搜索时间偏差的 RMSE 型目标函数，$\|\cdot\|_2$ 表示二维欧氏距离。使用平方平均的原因是：时间偏差应让整段重叠轨迹整体对齐，而不是只让某一个点最近；平方项会惩罚错位较大的轨迹段，开方后又保留米作为单位，便于与定位误差尺度直接比较。

表 4　附件 1 时间对齐与 10 Hz 输出结果

数据	Δ/s	重叠起点/s	重叠终点/s	10Hz 点数	对齐前 RMSE/m	对齐后 RMSE/m
附件 1	-198.4317	270.4000	970.7500	7004	5.6981	9.06×10^{-8}

表 4 中对齐前 RMSE 采用未修正时间轴 $\Delta = 0$ 的重叠区间计算，对齐后 RMSE 采用最优 Δ 计算，因此两者口径明确且不相同。

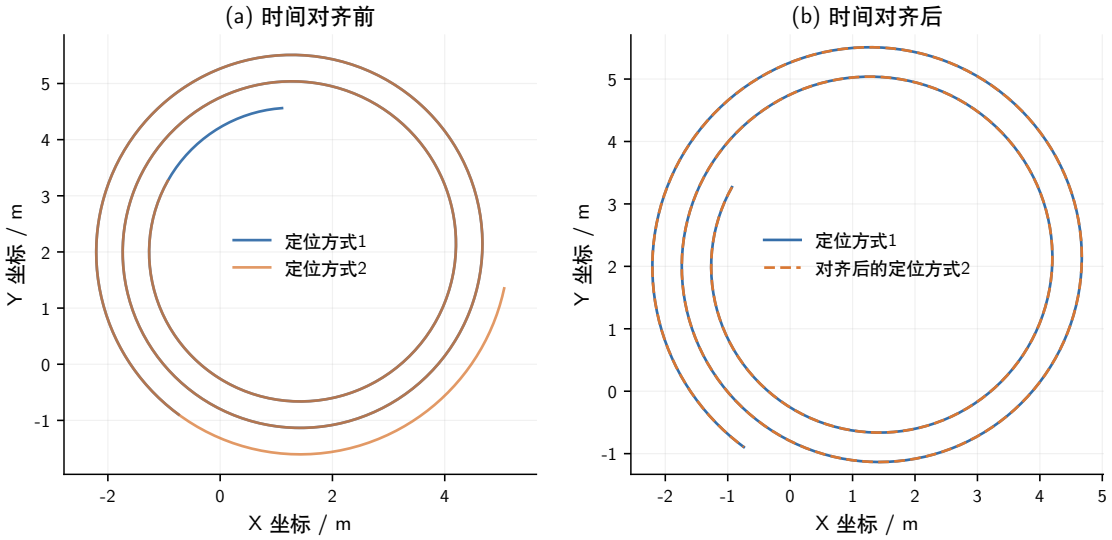

图 2　附件 1 时间对齐前后的两源轨迹对比

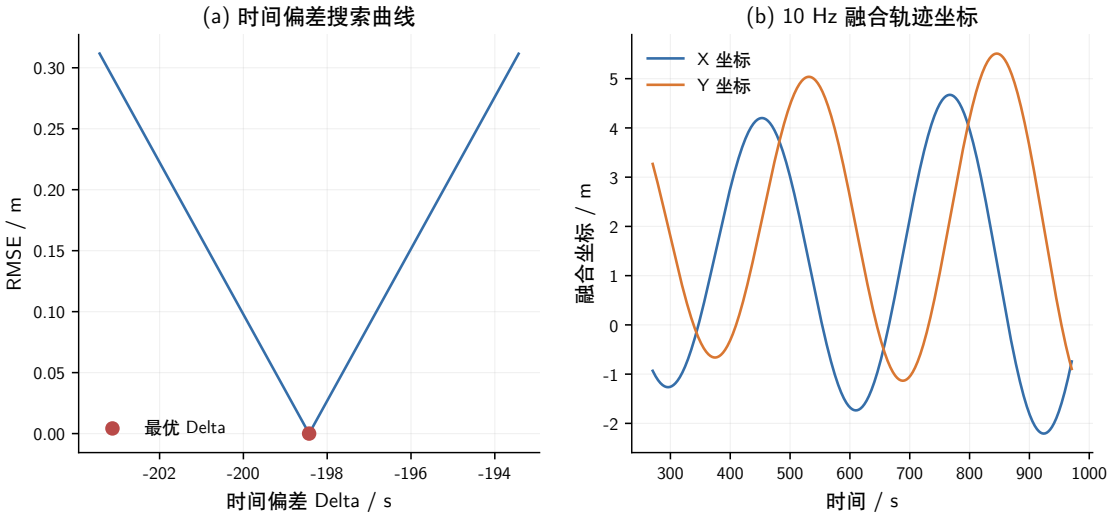

图 3　附件 1 RMSE–Δ 搜索曲线与 10 Hz 融合输出

3.3　结果分析

$J(\Delta)$ 在图 3 的最优偏差附近形成清晰极小值，说明时间偏差由整段轨迹的整体几何一致性确定，而不是某个局部点的偶然匹配。对齐后的两源轨迹几乎重合，因附件 1 无噪声，融合轨迹本质上是对同一真实轨迹的 10 Hz 重采样。

图 2 中，对齐前两条曲线形状相近但沿轨迹方向错开；若直接计算空间差，时间滞后会被误判为空间测量偏差。对齐后两源曲线重叠，表明附件 1 不存在需要继续估计的固定平移偏差。图 3 右侧给出的 10 Hz 坐标序列是后续速度、加速度和任务窗口判断的基础，因此问题一输出的是可复现的公共时间轨迹文件。时间参数化模型适用于两源观测同一连续轨迹且时间偏差近似为常数的情形；若存在分段时钟漂移或传感器中途重

同步，则单一 Δ 只能给出平均对齐结果，后续应扩展为分段时间偏差或动态时间规整模型。

四、问题二：含噪轨迹的固定偏差识别与状态空间融合

4.1 问题分析与模型选择

附件 2 同时存在时间偏差、方式 2 相对方式 1 的固定平移型相对偏差以及随机观测噪声。若仅做时间对齐，空间偏差仍会残留；若仅做平移校正，则可以降低位置残差，但难以得到稳定的速度和加速度序列。问题四中的候选事件判定直接依赖运动学量，因此问题二需要同时处理偏差识别与状态平滑。

基于这一要求，先用鲁棒中位数估计固定平移偏差，再引入卡尔曼滤波与 RTS 平滑分离运动状态、固定偏差和随机测量误差。RTS 在这里的主要作用是稳定速度、加速度等运动学量，而不是显著降低位置 RMSE。

4.1.1 附件 2 误差结构的初步判断

表 5　附件 1 与附件 2 对齐及误差结构对比

数据	主要误差来源	Δ/s	对齐前 RMSE/m	校正后 RMSE/m	固定偏差模长/m	10 Hz 点数
附件 1	时间错位	-198.4317	5.6981	9.06×10^{-8}	0.0000	7004
附件 2	时间错位、固定偏差、随机噪声	-50.4947	3.9970	0.7315	3.9281	6900

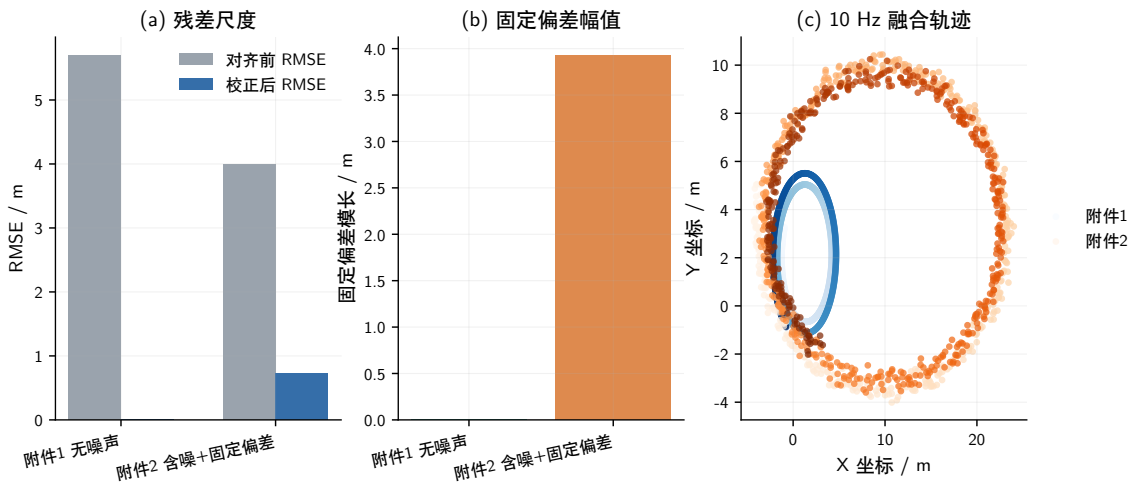

图 4　附件 1 与附件 2 对齐误差、固定偏差和 10 Hz 融合轨迹对比

表 5 和图 4 先给出一个直接判断：附件 1 的误差几乎完全来自时间错位，对齐后即可重合；附件 2 在完成时间对齐后仍保留 3.9281 m 量级的固定偏差和 0.7315 m 量级的剩余残差，因此不能只靠时间配准解释。

因此，附件 2 不能沿用附件 1 的单一时间偏移模型。后续模型在时间对齐之后继续估计方式 2 相对方式 1 的稳定空间偏移，并把剩余误差作为随机测量误差处理，以便输出更平滑的运动学状态。

4.2 模型构建与求解流程

将附件 2 观测写为

$$\boldsymbol{X} = [x, y, v_x, v_y, b_x, b_y]^\top, \quad \boldsymbol{X}_{k+1} = F_k \boldsymbol{X}_k + \boldsymbol{w}_k. \tag{3}$$

其中状态转移矩阵采用常速度、常偏差形式

$$F_k = \begin{bmatrix} 1 & 0 & \Delta t_k & 0 & 0 & 0 \\ 0 & 1 & 0 & \Delta t_k & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}, \qquad \Delta t_k = t_{k+1} - t_k. \tag{4}$$

其中 x, y 是机器人真实位置，v_x, v_y 是速度分量，b_x, b_y 是方式 2 相对方式 1 的固定平移偏差，$\boldsymbol{w}_k$ 表示过程扰动。把 $\boldsymbol{b}$ 放进状态向量的原因是：固定偏差在时间上近似不变，但其估计又应和位置状态同时利用两源观测来更新。随机观测噪声不放入状态，而由 R_1, R_2 描述，这是卡尔曼滤波中区分状态不确定性和测量不确定性的基本做法 [3]。方式 1 作为参考源，其观测矩阵为

$$H_1 = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \end{bmatrix}. \tag{5}$$

方式 2 含固定相对偏差，其观测矩阵为

$$H_2 = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 \end{bmatrix}. \tag{6}$$

随机观测误差由 R_1, R_2 表示，过程不确定性由 Q 表示。该写法与参考文献中的卡尔曼滤波方程一致，但本文不做未来轨迹预测，而是用滤波与 RTS 平滑获得同一时段的稳健状态估计 [4]。

滤波递推采用标准预测–更新形式：

$$\hat{\boldsymbol{X}}_{k|k-1} = F_k \hat{\boldsymbol{X}}_{k-1|k-1}, \tag{7}$$

$$P_{k|k-1} = F_k P_{k-1|k-1} F_k^\top + Q, \tag{8}$$

$$K_k = P_{k|k-1} H_k^\top (H_k P_{k|k-1} H_k^\top + R_k)^{-1}, \tag{9}$$

$$\hat{\boldsymbol{X}}_{k|k} = \hat{\boldsymbol{X}}_{k|k-1} + K_k(\boldsymbol{z}_k - H_k \hat{\boldsymbol{X}}_{k|k-1}). \tag{10}$$

其中 H_k 根据当前观测来源取 H_1 或 H_2。RTS 平滑在滤波后沿时间反向递推，用后续观测修正前一时刻状态，从而获得更平滑的 10 Hz 位置、速度和偏差序列。

上述方程中，$\hat{\boldsymbol{X}}_{k|k-1}$ 是利用上一时刻状态预测得到的先验状态，$P_{k|k-1}$ 是先验协方差，K_k 是卡尔曼增益，$\boldsymbol{z}_k - H_k\hat{\boldsymbol{X}}_{k|k-1}$ 是观测创新项。创新项刻画当前观测与模型预测的偏离程度；R_k 控制观测权重，Q 控制状态随时间变化的自由度。后文的 Q/R 灵敏度分析用于检查固定偏差估计是否会随这些参数明显漂移。

4.3 结果分析

表 6 附件 2 鲁棒基准、卡尔曼滤波与 RTS 平滑结果

方法	Δ/s	b_x/m	b_y/m	对齐前 RMSE/m	校正后 RMSE/m
RobustMedian	-50.4947	3.4800	-1.8221	3.9970	0.7315
KalmanFilter	-50.4947	3.4809	-1.8284	3.9970	0.7314
KalmanRTS	-50.4947	3.4754	-1.8337	3.9970	0.7314

表 6 中，RobustMedian、卡尔曼滤波和 RTS 平滑的 RMSE 非常接近，说明附件 2 的主要位置误差已由固定平移项解释；状态空间模型的增益主要体现在把随机观测误差纳入 R_1, R_2，并将固定偏差写入状态变量 $\boldsymbol{b}$。

鲁棒中位数配准在这里作为基准模型使用：它先用残差中位数估计固定平移，能降低少数异常点对偏差估计的影响；卡尔曼滤波和 RTS 平滑则在此基础上提供状态空间解释。三者 RMSE 接近，说明“固定偏差 + 随机噪声”的误差结构已能较好解释附件 2 的观测差异。

表 7 附件 2 随机噪声方差与残差诊断

指标	含义	数值	单位
$\mathrm{Var}(r_x)$	固定偏差校正后 x 残差方差	0.2566	m^2
$\mathrm{Var}(r_y)$	固定偏差校正后 y 残差方差	0.2783	m^2
σ_r	残差合成尺度 $\sqrt{\mathrm{Var}(r_x)+\mathrm{Var}(r_y)}$	0.7314	m
R_{base}	灵敏度分析使用的基础观测噪声方差	0.2675	m^2
RTS 相对鲁棒基准 RMSE 改善	验证模型并非大幅降 RMSE	0.0001	m

表 7 给出了校正后随机残差的方差尺度。x、y 两方向残差方差接近，说明随机噪声没有明显单轴异常；RTS 相对鲁棒基准的 RMSE 改善只有 0.0001 m，进一步支持“验证型增强模型”的定位。

残差方差表的作用是把“随机噪声”从口头判断变成可度量参数。后续任务调度使用的速度和加速度来自平滑轨迹，若附件 2 残差方差很大，则平滑状态的运动学量也应

降低信任度；本题中残差合成尺度约 0.7314 m，属于可被固定偏差校正后保留的随机测量误差范围。

表 8　附件 2 Q/R 参数灵敏度

配置	q_p	q_v	q_b	$r_{1x}=r_{2x}$	b_x/m	b_y/m
low_process	0.002	0.010	10^{-7}	0.2675	3.4807	-1.8283
balanced	0.010	0.050	10^{-6}	0.2675	3.4803	-1.8275
smooth_bias	0.030	0.100	10^{-8}	0.3210	3.4809	-1.8284

三组参数下 Δ 均为 -50.4947 s，固定偏差变化小于 0.01 m。在该数据集上，卡尔曼滤波与 RTS 平滑并未显著改变位置 RMSE，其主要贡献是把运动状态、固定偏差和随机测量误差放入同一状态空间口径，并为后续速度、加速度计算提供平滑状态轨迹。

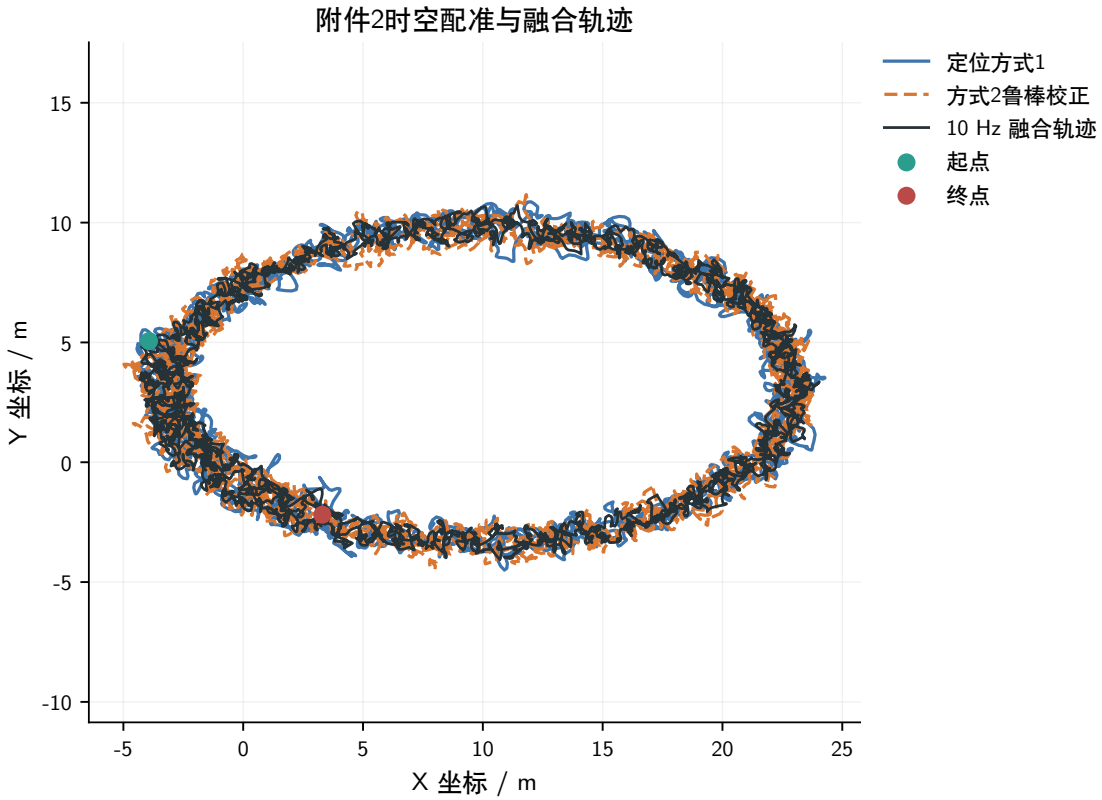

图 5　附件 2 对齐前、鲁棒校正后与 RTS 平滑轨迹对比

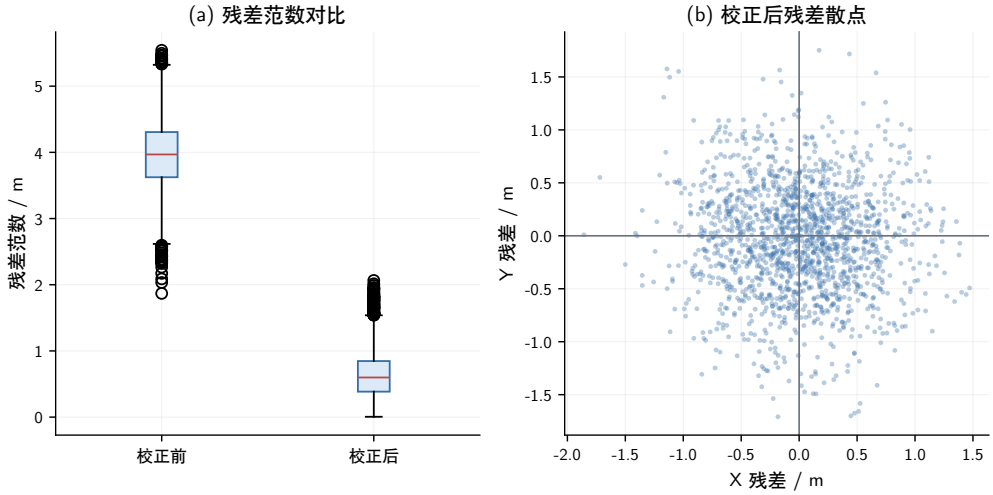

图 6　附件 2 残差分布诊断

图 5 中，校正前两源轨迹整体呈近似平行错位，鲁棒校正后明显贴合，RTS 曲线主要表现为局部平滑。图 6 中，固定偏差校正后的残差范数明显收缩，二维残差集中在原点附近。两幅图对应的结论是一致的：附件 2 中的主要系统差异可以用方式 2 相对方式 1 的稳定空间偏移解释，其余部分进入 R_1, R_2 作为随机测量误差。

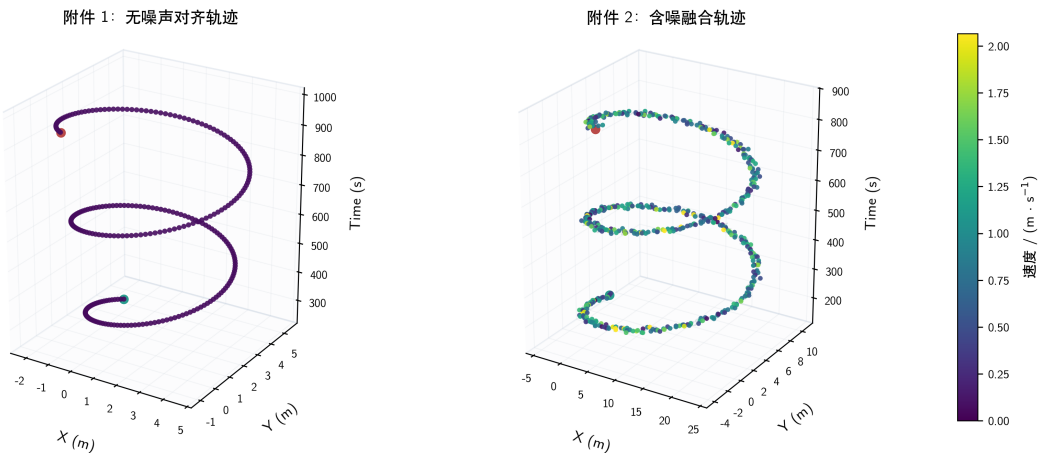

图 7　附件 1 与附件 2 时间参数化三维轨迹及速度散点

图 7 中，横、纵坐标为空间位置 X, Y，竖直方向为时间 t，轨迹的空间位置、时间推进和速度变化同时被保留下来。相同空间区域内，如果轨迹颜色对应较高速度，机器人只是快速通过，射击和拍照准备窗口通常较短；若局部轨迹段速度较低且持续时间较长，则更容易形成可执行事件。附件 1 给出了无噪声条件下清晰的时间推进关系，附件 2 则显示了含噪融合轨迹中的速度分布。后续候选事件生成因此需要依赖 10 Hz 时间轨迹及其速度、加速度状态，而不能退化为目标点到平面轨迹的最近距离匹配。

表 9　附件 2 有限差分与 RTS 状态运动学量对比

方法	速度 95 分位/(m/s)	速度总变差	加速度 95 分位/(m/s^2)	加速度总变差
参考源有限差分	2.3017	3057.83	13.7451	23045.26
RTS 平滑状态	1.8138	2622.21	11.4148	21030.99

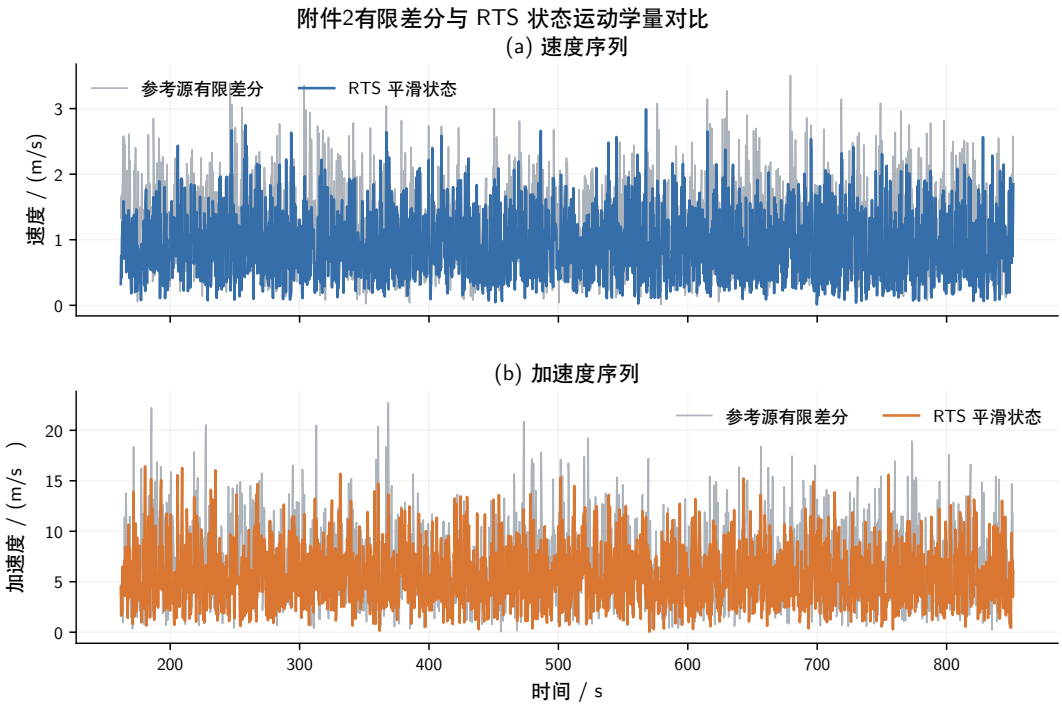

图 8　附件 2 有限差分与 RTS 状态速度、加速度序列对比

4.3.1　运动学量稳定性与后续作用

为保持速度、加速度口径与后续任务判定一致，本文统一在 10 Hz 平滑轨迹上采用中心差分计算运动学量。设公共时间步长为 $\Delta t = 0.1$ s，则

$$v_x(t_k) = \frac{x(t_{k+1}) - x(t_{k-1})}{2\Delta t}, \qquad v_y(t_k) = \frac{y(t_{k+1}) - y(t_{k-1})}{2\Delta t}, \tag{11}$$

$$a_x(t_k) = \frac{v_x(t_{k+1}) - v_x(t_{k-1})}{2\Delta t}, \qquad a_y(t_k) = \frac{v_y(t_{k+1}) - v_y(t_{k-1})}{2\Delta t}, \tag{12}$$

并定义

$$v(t_k) = \sqrt{v_x^2(t_k) + v_y^2(t_k)}, \qquad a(t_k) = \sqrt{a_x^2(t_k) + a_y^2(t_k)}. \tag{13}$$

若直接对含噪坐标做有限差分，高频噪声会被放大，速度和加速度阈值附近的候选事件容易被误删或误保留，因此这里采用 RTS 平滑状态导出的运动学量作为问题四的统一输入。

表 9 和图 8 显示，RTS 平滑对位置 RMSE 的影响很小，但能明显压低速度和加速度序列的高分位抖动。与参考源有限差分相比，RTS 状态的速度 95 分位由 2.3017 m/s

降至 1.8138 m/s，加速度 95 分位由 13.7451 m/s^2 降至 11.4148 m/s^2。由于问题四的候选事件剪枝直接依赖 $v(t)$ 和 $a(t)$，有限差分放大的高频噪声会影响边界事件判定；RTS 状态降低高分位波动后，这些判定更稳定。问题二的结果也说明，该状态空间模型适用于两源观测同一运动目标、方式 2 相对方式 1 的偏差在观测时段内近似稳定的场景；若偏差随温度、姿态或通信时延快速变化，固定 b_x, b_y 状态将不足以解释残差，后续应扩展为时变偏差、分段偏差或带外部影响因子的状态模型。

五、问题三：实际测量数据的残差结构检验

5.1 问题分析与模型选择

附件 3 属于实际测量数据，不能直接假设误差结构仍然是固定平移偏差。残差中可能存在局部变化，但这些变化未必能够稳定外推到其他时间区间。若直接使用复杂模型强行校正轨迹，训练误差可能下降，后续任务判定却会受到不稳定偏差的影响。

基于这一点，本节采用 M0–M4 模型族比较不同偏差结构，并用时间块交叉验证检验其泛化能力。若复杂模型在测试时间块上不能稳定降低误差，则不把它写入强校正轨迹，而只保留其残差统计特征，供问题四进行风险复核。

5.2 模型构建与求解流程

附件 3 先完成时间对齐，得到 $\Delta = 367.8843$ s。原始插值残差 RMSE 约 2.750 m；表 10 只报告平滑状态残差上的模型内拟合误差，用于观察复杂模型是否吸收局部结构。真正决定是否强校正的外推指标，统一采用表 11 的 5 折时间块 CV-RMSE。构造特征

$$\boldsymbol{\psi} = (t, x, y, v, a, \cos\theta, \sin\theta, \kappa),$$

并比较 M0 无偏差、M1 固定平移、M2 线性时变、M3 Ridge 状态相关线性映射和 M4 Kernel Ridge 非线性映射。Ridge 与 Kernel Ridge 都属于带正则项的回归模型，前者偏向解释线性关系，后者允许非线性但更容易过拟合 [8,9]。时间序列按顺序分为 5 个连续块，每次用 4 块训练、1 块测试，标准化参数只由训练块计算。

采用时间块交叉验证而非随机打乱交叉验证，是因为附件 3 残差可能具有时间相关性。若随机打乱，训练集和测试集会同时包含相邻时刻，模型可能只是在记忆局部趋势，测试误差会被过度乐观地低估。时间块划分要求模型用一段时间中的规律解释另一段时间中的残差，更接近后续任务规划中“用历史残差评估未来窗口风险”的需求。重抽样思想也在这里体现为对时间块稳定性的检验，而不是只相信一次全样本拟合 [7]。

表 10　附件 3 系统偏差结构模型内拟合比较

模型	含义	复杂度	训练 RMSE/m	全样本残差 RMSE/m	训练改善
M0	无系统偏差	0	1.9456	1.9456	0.0000
M1	固定平移偏差	2	1.9402	1.9402	0.0000
M2	线性时变偏差	4	1.9379	1.9379	0.0012
M3	状态相关线性偏差	16	1.9159	1.9159	0.0126
M4	Kernel Ridge 非线性偏差	1500	1.6217	1.6217	0.1642

表 10 中，M3、M4 的训练 RMSE 明显下降，尤其 M4 的模型内拟合误差降至 1.6217 m。这一现象只能说明复杂模型有能力吸收残差纹理，不能直接说明该纹理具有跨时间稳定性。若把模型内拟合误差误当作外推误差，就会把局部波动写入名义轨迹。

为把“是否强校正”写成明确规则，定义复杂模型相对固定偏差模型 M1 的外推收益

$$\Delta_{\mathrm{CV}}(M_j) = \frac{\text{CV-RMSE}(M_1) - \text{CV-RMSE}(M_j)}{\text{CV-RMSE}(M_1)}.$$

若 $\Delta_{\mathrm{CV}}(M_j) > 0$，说明模型 M_j 在时间块外推中优于固定偏差模型，可以考虑将其作为强校正；若 $\Delta_{\mathrm{CV}}(M_j) \leq 0$，则复杂结构没有外推收益，只能作为风险来源进入后续复核。

表 11　附件 3 5 折时间块交叉验证误差

模型	Fold1	Fold2	Fold3	Fold4	Fold5	平均 CV-RMSE/m
M0	2.8014	3.1583	2.5308	2.6818	2.5363	2.7417
M1	2.8131	3.1453	2.5304	2.6674	2.5402	2.7393
M2	2.8046	3.1511	2.5305	2.6801	2.5497	2.7432
M3	3.9585	3.2534	2.6004	2.6689	2.6633	3.0289
M4	2.8466	3.3106	2.8010	2.7568	2.6031	2.8636

表 11 中，M3 的误差主要集中在前两折放大，M4 则在第 2–4 折持续劣于 M1。这种分折结果表明，复杂模型捕捉到的残差结构带有明显时间段特征，并没有形成全时段稳定映射。继续提高模型复杂度可以降低模型内拟合误差，却不能改善时间块外推误差；因此后续名义轨迹采用 M1，其余残差信息进入鲁棒场景。

按上述判据，M2、M3、M4 的 Δ_{CV} 分别约为 -0.14%、-10.57%、-4.54%，均不满足强校正条件。训练误差与外推误差方向相反，表明 M3/M4 主要吸收了局部时间段的残差纹理，而不是全时段稳定偏差。若将这类结构叠加到轨迹，问题四中靠近距离、速度和 FOV 边界的候选事件会被系统性移动。该判断依赖当前无外部高精度真值标定的条件；若后续获得参考轨迹，应重新检验时变偏差模型。

5.2.1 残差场景构造与风险量化

残差结构进入问题四时不直接修正轨迹，而是用于候选事件的鲁棒性复核。设附件 3 平滑状态残差为

$$\boldsymbol{r}(t) = \boldsymbol{p}_2(t-\Delta) - \boldsymbol{p}_1(t) - \hat{\boldsymbol{b}},$$

将时间序列分成 $K=5$ 个连续块，第 j 个时间块的残差均值和协方差分别为 $\bar{\boldsymbol{r}}_j$ 与 Σ_j。正文采用简化代表场景，对每个块构造位置扰动集合

$$\mathcal{U}_j = \{\bar{\boldsymbol{r}}_j,\ \bar{\boldsymbol{r}}_j \pm \lambda(\sqrt{\Sigma_{j,11}}, 0)^\top,\ \bar{\boldsymbol{r}}_j \pm \lambda(0, \sqrt{\Sigma_{j,22}})^\top\},$$

其中 λ 取 1 表示一倍标准差扰动。该构造保留了残差的时间分块均值，又用两个坐标方向的标准差表示局部不确定范围，属于鲁棒优化中用代表性扰动场景近似不确定集的处理方式 [13]。若残差 x/y 相关性较强，可把该集合扩展到协方差主方向：设 $\Sigma_j = U_j \Lambda_j U_j^\top$，则加入

$$\bar{\boldsymbol{r}}_j \pm \lambda\sqrt{\lambda_{j,1}}\boldsymbol{u}_{j,1}, \qquad \bar{\boldsymbol{r}}_j \pm \lambda\sqrt{\lambda_{j,2}}\boldsymbol{u}_{j,2}.$$

本文实际计算采用坐标轴扰动，因此所得风险评估是可复现的工程近似；主轴扰动可作为进一步保守复核。

对候选事件 e 和扰动场景 s，将轨迹位置改写为

$$\boldsymbol{p}_f^{(s)}(t_e) = \boldsymbol{p}_f(t_e) + \boldsymbol{u}_s, \qquad \boldsymbol{u}_s \in \mathcal{U}_j,$$

并重新计算目标距离、速度、加速度和拍照方位角裕度。记这些单项裕度的最小值为

$$m_e^{(s)} = \min\{m_{e,d}^{(s)}, m_{e,v}^{(s)}, m_{e,a}^{(s)}, m_{e,\theta}^{(s)}\},$$

则事件的最坏裕度和场景通过率定义为

$$m_e^{\text{worst}} = \min_s m_e^{(s)}, \qquad \rho_e = \frac{1}{|\mathcal{U}|}\sum_s I(m_e^{(s)} \geq 0).$$

当 m_e^{worst} 接近 0 或 ρ_e 偏低时，事件在鲁棒复核表中被标记为风险任务。主方案 A 的 MILP 目标中采用基于名义裕度的分段风险惩罚 R_{risk}；保守方案 B 则进一步施加 $m_e^{\text{nom}} \geq 0.03$ 的筛选阈值。本文不直接把 m_e^{worst} 写入硬约束，是为了避免在无外部真值条件下过度保守；保守方案 B 通过名义裕度筛选和风险权重提高，间接吸收扰动风险。

时间块 CV 与鲁棒场景的适用前提是：残差中存在可观察结构，但缺少外部真值证明其稳定外推能力。本文的扰动场景来自观测残差而非独立标定实验，因此更适合作为工程风险吸收机制；若后续获得高精度参考轨迹或传感器物理参数，应重新建立可解释的时变系统偏差模型。

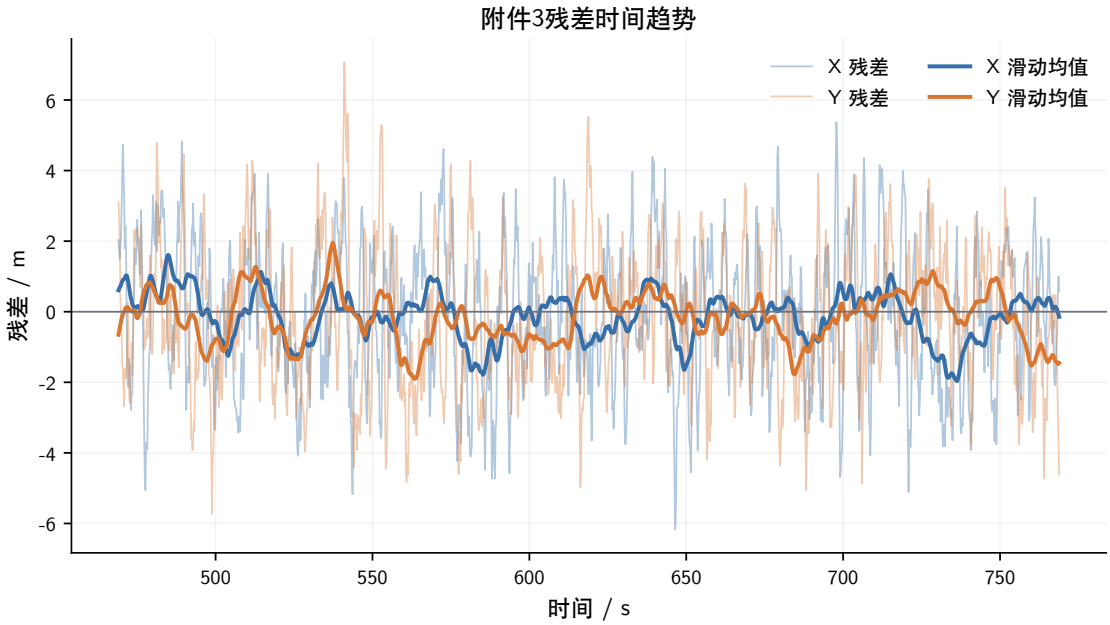

图 9　附件 3 残差随时间变化及趋势线

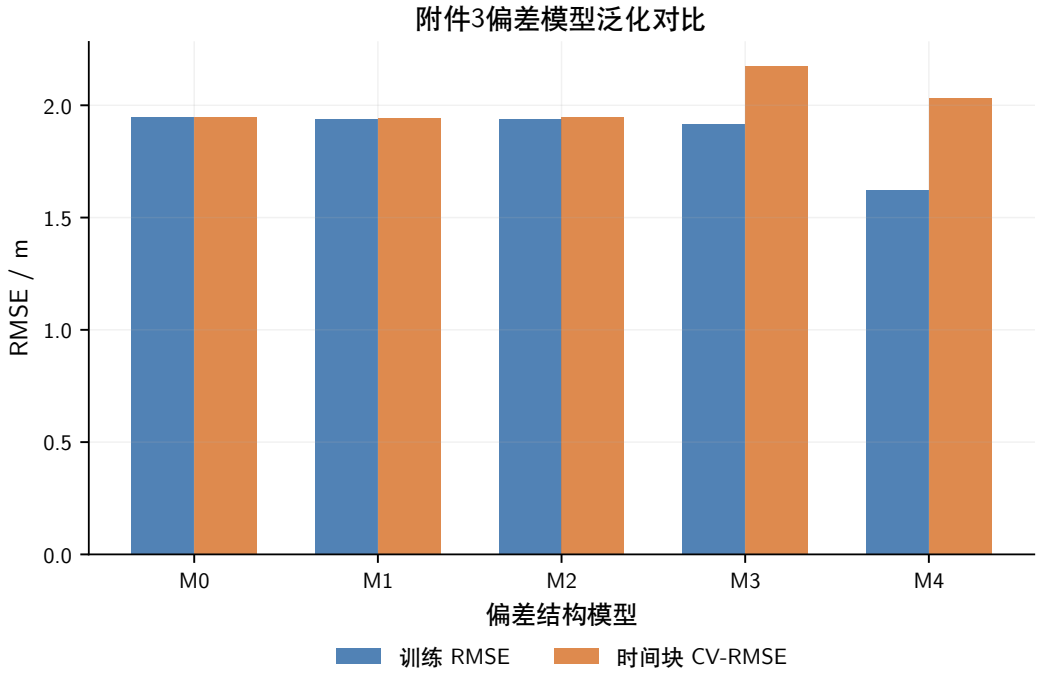

图 10　附件 3 M0–M4 训练 RMSE 与时间块 CV-RMSE 对比

5.3　结果分析

x、y 残差在图 9 中都表现出明显的分段波动，且局部偏移方向会随时间改变，这意味着附件 3 中的误差不是“再加一个常数偏移”就能解释的静态结构，而更接近随时段变化的弱非平稳扰动。问题在于，这类结构虽然存在，却没有表现出足够稳定的跨时段重复性。

图 10 与表 11 共同说明，M3、M4 的优势只停留在训练段内，一旦换到新的时间块，误差就重新放大。其机制在于：复杂模型会优先追踪局部残差纹理，但这些纹理并不代表未来时段仍然成立的系统偏差。若据此强行修轨迹，就等于把“不稳定的信息”提前写死到后续任务窗口判定里。基于这一判断，问题三不输出强校正轨迹，而输出事件级风险场景；这一步把“残差难以可靠解释”的问题，转化成了“边界任务需要谨慎选择”的约束信息。

六、问题四：固定轨迹下的射击与拍照任务优化

6.1 问题分析与模型选择

问题四的轨迹已经由前 3 问确定，决策对象是固定轨迹上的任务事件，而不是路径规划。射击候选数量多，但对同一目标多次射击的边际收益递减；拍照候选数量少，但一次拍照可能覆盖多个目标，具有集合覆盖价值。两类任务共享同一时间轴，并分别受到 1.5 s 和 0.5 s 准备窗口约束，时间冲突十分密集。

在这种条件下，局部贪心难以同时处理覆盖收益、准备窗口冲突和风险偏好，因此采用事件级 MILP 统一建模。问题三提供的分块残差统计量继续用于候选事件的扰动复核，从而在任务优化阶段显式考虑名义裕度、最坏裕度和场景通过率。

6.2 模型构建与求解流程

问题四的 MILP 可以拆成三层。第一层决定候选事件是否被选取，即 q_e, x_e, z_e；第二层处理时间准备窗口和拍照角度互斥，即 $\mathcal{C}_T$ 与 $\mathcal{C}_\theta$；第三层把射击命中、拍照覆盖和风险裕度写入目标函数，即 $y_{k,m}, u_p$ 及 R_{risk}。下面先生成候选事件，再给出这三层变量与约束。

6.2.1 候选事件生成

射击事件 $e = (k, t)$ 要求在 $[t - 1.5, t]$ 内满足 $5 \le d_k \le 30$、$v \le 2$、$a \le 1.5$。拍照事件 $e = (t, \alpha)$ 要求在 $[t - 0.5, t]$ 内满足 $10 \le d_p \le 40$、$v \le 1.5$、$a \le 1.5$，并且目标方位角落在 FOV 内，其中 α 表示相机朝向角。主结果取 FOV=45°，另做 $30°, 60°$ 灵敏度。

机器人轨迹已经由前述融合模型确定，因此任务被离散为候选事件集合。这里对射击目标和拍照目标统一整理编号、坐标和任务类型，再对每个潜在执行时刻记录准备开始时刻、距离、速度、加速度、FOV 约束和多角度信息。射击候选由“目标编号–执行时刻”组成，拍照候选由“执行时刻–相机朝向–覆盖目标集合”组成；这些字段既用于 MILP 约束，也用于最终逐目标解释和 Gantt 图展示。

表 12　候选事件生成、剪枝与冲突统计

阶段	原始检查数	可行/边数	剪枝数	说明
射击枚举	53730	981	52749	距离、速度、加速度、1.5s 窗口
拍照枚举	10782	47	10735	距离、FOV、速度、加速度、0.5s 窗口
联合 MILP 候选池	1028	1028	0	主方案不设额外裕度筛除
时间冲突边	527878	43744	0	窗口重叠约束
拍照角度冲突边	1081	467	0	同目标角差小于 60°

表 12 反映出明显的任务资源不对称性。射击候选数量大，说明射击机会主要受时间冲突限制；拍照候选只有 47 个，说明拍照机会首先受 FOV、方向角和运动学窗口共同压缩。43744 条时间冲突边说明可行事件集中在少数轨迹片段内，准备窗口高度重叠，因此联合优化中需要为少量高价值拍照事件保留时间空间。

候选统计还解释了“为什么先分别做纯射击和纯拍照，再做联合方案”。纯射击方案用于判断射击目标在固定轨迹上的最大可达性，纯拍照方案用于判断 FOV 和方向角导致的拍照稀缺性，联合方案则在二者之间重新分配时间窗口。若没有前两个单任务基准，联合方案的收益损失或收益提升就没有可比较对象。

6.2.2　冲突边与覆盖关系构造

联合优化的关键不是单个候选事件是否可行，而是候选事件之间如何争夺同一时间窗口和同一拍照目标。为此，先在候选池层面预先构造两类冲突边和一类覆盖关系。

第一类是时间冲突边 $\mathcal{C}_T$。若两个事件的准备窗口 $W_i = [s_i, t_i]$ 与 $W_j = [s_j, t_j]$ 重叠，则二者不能同时执行，记为 $(i, j) \in \mathcal{C}_T$，并在 MILP 中写成 $q_i + q_j \leq 1$。由于射击准备时长为 1.5 s、拍照准备时长为 0.5 s，大量候选事件会在少数局部时间段密集堆叠，正文中的 43744 条时间冲突边正是这一资源竞争的直接量化结果。

第二类是拍照角度冲突边 $\mathcal{C}_\theta$。若两个拍照事件都覆盖同一拍照目标，且该目标在两次观测中的圆周角差

$$\delta_\theta(i, j) = \min(|\theta_i - \theta_j|, 360^\circ - |\theta_i - \theta_j|)$$

小于 60°，则这两次观测不再视为有效多角度信息，故加入约束 $z_i + z_j \leq 1$。这样做的目的不是限制拍照次数本身，而是避免模型把几乎同向的重复观测误算为“多角度收益”。

覆盖关系则由拍照事件的目标集合 C_e 给出。若拍照事件 e 的 FOV 扇形同时覆盖多个目标，则这些目标都与事件 e 建立连接，用于构造覆盖变量 u_p 和多目标同拍奖励 B_{multi}。因此，冲突边负责描述“哪些事件不能同时做”，覆盖集合负责描述“一个事件能带来哪些目标收益”，两者共同决定了问题四中事件级优化的结构。

6.2.3 MILP 变量、约束与目标函数

对每个候选事件定义统一变量 $q_e \in \{0,1\}$，并用

$$q_e = \begin{cases} x_e, & e \in \mathcal{E}^S, \\ z_e, & e \in \mathcal{E}^P \end{cases}$$

连接射击变量和拍照变量。若 e 是射击事件，则 $x_e = q_e, z_e = 0$；若 e 是拍照事件，则 $z_e = q_e, x_e = 0$。对射击目标 k，令

$$n_k = \sum_{e \in \mathcal{E}_k^S} x_e, \qquad \sum_{m=1}^{M_k} y_{k,m} = n_k, \qquad y_{k,m} \le y_{k,m-1} \ (m \ge 2),$$

其中 $y_{k,m}$ 表示目标 k 是否获得第 m 次射击。对拍照目标 p，覆盖变量满足

$$u_p \le \sum_{e: p \in C_e} z_e, \qquad z_e \le u_p \quad (p \in C_e),$$

从而 $u_p = 1$ 当且仅当至少一个被选拍照事件覆盖该目标。第 m 次射击边际收益为

$$g_m = 0.85 \cdot 0.15^{m-1}, \quad P_k(n_k) = 1 - 0.15^{n_k}.$$

其中指数 $m-1$ 表示同一目标第 m 次射击的边际收益递减，指数 n_k 表示目标 k 的累计射击次数，反映多次独立射击下至少一次命中的概率。若两个事件准备窗口重叠，则 $q_i + q_j \le 1$；若同一拍照目标两次观测方向的圆周角差小于 $60°$，则 $z_i + z_j \le 1$。

单目标最大射击次数取 $M_k = 5$，依据有三点。第一，边际收益满足 $g_m = 0.85 \times 0.15^{m-1}$，到第 5 次时仅剩 $g_5 = 4.30 \times 10^{-4}$，相对于第一次射击的收益已经可以忽略。第二，累计命中概率到 5 次时达到 $P_k(5) = 1 - 0.15^5 = 0.999924$，继续增加射击次数几乎不会改变目标完成层的效用。第三，从候选事件统计看，可执行时间窗口的主要矛盾来自跨目标冲突而非单目标射击次数不足，因此把 M_k 再放大只会增加槽位变量数量，而不会实质改善最优解质量。

核心约束组 正文只保留决定模型结构的核心约束，完整集合与变量说明见附录。记 $\mathcal{E} = \mathcal{E}^S \cup \mathcal{E}^P$，事件准备窗口为 $W_e = [s_e, t_e]$。联合任务调度可概括为

$$q_e = x_e \ (e \in \mathcal{E}^S), \qquad q_e = z_e \ (e \in \mathcal{E}^P), \tag{14}$$

$$n_k = \sum_{e \in \mathcal{E}_k^S} x_e, \qquad \sum_{m=1}^{M_k} y_{k,m} = n_k, \qquad y_{k,m} \le y_{k,m-1}, \tag{15}$$

$$u_p \le \sum_{e: p \in C_e} z_e, \qquad u_p \ge z_e \quad (p \in C_e), \tag{16}$$

$$q_i + q_j \le 1 \quad ((i,j) \in \mathcal{C}_T), \qquad z_i + z_j \le 1 \quad ((i,j) \in \mathcal{C}_\theta), \tag{17}$$

$$q_e \le I(m_e^{\text{nom}} \ge 0), \qquad q_e, x_e, z_e, y_{k,m}, u_p \in \{0,1\}. \tag{18}$$

其中 $\mathcal{C}_T$ 控制准备窗口互斥，$\mathcal{C}_\theta$ 控制同一拍照目标的近同向重复观测，$q_e \le I(m_e^{\mathrm{nom}} \ge 0)$ 表示候选事件必须满足名义裕度非负。保守方案 B 仅在候选池入口额外施加 $m_e^{\mathrm{nom}} \ge 0.03$ 的硬筛选，而 m_e^{worst} 与 ρ_e 仍作为解后鲁棒性复核指标。

目标函数为

$$\max\ M_1(E_{\mathrm{shoot}} + C_{\mathrm{photo}}) + M_2 O_{\mathrm{photo}} + M_3 B_{\mathrm{multi}} + M_4 \sum_e m_e q_e - M_5 R_{\mathrm{risk}}. \tag{19}$$

其中

$$E_{\mathrm{shoot}} = \sum_{k,m} g_m y_{k,m}, \tag{20}$$

$$C_{\mathrm{photo}} = \sum_p u_p, \tag{21}$$

$$O_{\mathrm{photo}} = \sum_e |C_e| z_e, \tag{22}$$

$$B_{\mathrm{multi}} = \sum_e \max(0, |C_e| - 1) z_e. \tag{23}$$

风险惩罚项明确写为

$$R_{\mathrm{risk}} = \sum_{e \in \mathcal{E}} \pi(m_e^{\mathrm{nom}}) q_e, \qquad \pi(m) = \begin{cases} 1.00, & m < 0.03, \\ 0.35, & 0.03 \le m < 0.08, \\ 0, & m \ge 0.08. \end{cases}$$

其中 m_e^{nom} 为事件 e 在名义轨迹下的最小归一化裕度。也就是说，越靠近约束边界的事件在目标函数中受到越强惩罚。本文统一采用如下 A/B 方案口径：A 方案在候选生成后仅要求 $m_e^{\mathrm{nom}} \ge 0$，并在目标函数中用 $\pi(m_e^{\mathrm{nom}})$ 进行软惩罚；B 方案则在同一候选定义下进一步施加 $m_e^{\mathrm{nom}} \ge 0.03$ 的硬筛选，并同步提高风险权重。两类方案都保留 m_e^{worst} 与 ρ_e 作为结果复核指标，但不把它们直接写成新的 Big-M 约束。该目标函数把“完成多少目标”放在第一层，把“拍照质量、多目标同拍、执行裕度和风险”放在第二层。这里的任务裕度统一按约束边界的归一化余量定义。对距离约束 $d_{\min} \le d_e \le d_{\max}$，定义

$$m_{e,d} = \min\left\{\frac{d_e - d_{\min}}{d_{\max} - d_{\min}}, \frac{d_{\max} - d_e}{d_{\max} - d_{\min}}\right\}.$$

对速度和加速度上界约束，定义

$$m_{e,v} = 1 - \frac{v_e}{v_{\max}}, \qquad m_{e,a} = 1 - \frac{a_e}{a_{\max}}.$$

拍照事件另有角度裕度 $m_{e,\theta}$，表示目标方位角到 FOV 边界的归一化距离，最终名义裕度取

$$m_e^{\mathrm{nom}} = \min\{m_{e,d}, m_{e,v}, m_{e,a}, m_{e,\theta}\}.$$

因此，m_e^{nom} 越接近 0，事件越靠近某个约束边界；若其为负，则该候选事件在名义轨迹下不可行。

为避免后文对三个鲁棒指标出现混用，这里统一说明其分工：m_e^{nom} 负责名义轨迹下的候选筛选与风险惩罚，m_e^{worst} 负责报告扰动场景中的最坏裕度，ρ_e 负责报告事件在离散扰动场景中的通过比例。A/B 两个方案都在同一组候选定义上比较，只是对 m_e^{nom} 的容忍程度和风险权重不同，因此表格中若出现“最小裕度”，默认指 $\min_e m_e^{\mathrm{nom}}$；若出现“最坏裕度”或“场景通过率”，则分别指 $\min_e m_e^{\mathrm{worst}}$ 与所选事件的 ρ_e 平均值。

权重取 $M_1 = 1000, M_2 = 10, M_3 = 2, M_4 = 0.25$；主方案 $M_5 = 0.20$，保守方案 $M_5 = 3.00$。尺度合理性来自三点。第一，目标完成项 $E_{\mathrm{shoot}} + C_{\mathrm{photo}}$ 的最小有效变化为约 1，而附加拍照观测和多目标同拍的总量远小于 M_1，因此任何一个目标覆盖收益都不会被次级指标反向覆盖，近似实现目标完成数优先的字典序。第二，$M_2 = 10$、$M_3 = 2$ 只在目标覆盖数相同或接近时区分拍照质量。第三，裕度 m_e 已归一化到 $[0, 1]$ 附近，$M_4 = 0.25$ 不会压倒目标收益；风险惩罚 M_5 从 0.20 提高到 3.00 可形成 A/B 两类方案，但仍不改变约束矩阵的 0/1 结构，HiGHS 求解过程未出现数值不稳定。由 A/B 方案的求解结果可见，该 MILP 适用于固定轨迹上的任务选择，即机器人路径不再改变，只在候选时刻选择射击或拍照；在这一条件下，时间互斥、拍照覆盖、射击收益递减和鲁棒风险惩罚都可以用 0–1 变量和线性约束表达。若允许机器人绕行、停车等待或相机连续转动，则候选事件集合本身会随控制策略变化，模型应扩展为路径规划与事件调度的联合优化。

6.2.4 求解流程与 A/B 方案设置

表 13 联合任务优化算法伪代码

步骤	操作
1	在 10 Hz 融合轨迹上枚举射击目标–执行时刻组合，按距离、速度、加速度和 1.5 s 准备窗口剪枝。
2	枚举拍照执行时刻与相机方向，计算 FOV 内可覆盖目标集合 C_e，按距离、速度、加速度和 0.5 s 准备窗口剪枝。
3	对所有候选事件计算准备窗口冲突边；对同一拍照目标计算圆周角差，生成角度冲突边。
4	建立 $q_e, x_e, z_e, y_{k,m}, u_p$ 变量及连接约束，写入目标函数。
5	调用 HiGHS 求解 MILP；若设置保守方案，则先剔除低裕度事件并提高 M_5 后重新求解。
6	输出事件表、逐目标表、填表文件和收益–风险复核表。

表 14 MILP 带权目标值与求解规模

方案	目标值	gap	变量数	约束数	候选事件	风险权重
A 收益优先	14163.1600	0.0	1111	44294	1028	0.20
B 裕度优先	11451.9716	0.0	800	22583	717	3.00

表 14 中的目标值是求解器使用的带权目标函数值，包含 $M_1 = 1000$ 等尺度权重；后文“综合效用”则是 $E_{\text{shoot}} + C_{\text{photo}}$，用于方案解释与横向比较。两个 MILP 均由 HiGHS 返回最优状态，gap 为 0。B 方案变量和约束少于 A 方案，是因为保守方案先剔除了低裕度事件，减少了候选池和冲突边。A 方案适合作为收益最大化主方案；B 方案牺牲部分射击期望命中和拍照覆盖，换取更高场景通过率和更低风险事件数。

6.3 结果分析

6.3.1 MILP 规模与可解性分析

联合候选池包含 1028 个事件，但 MILP 的有效难度主要来自冲突边而不是变量数本身。时间冲突边有 43744 条，对应准备窗口互斥约束；拍照角度冲突边有 467 条，对应同一拍照目标多角度约束。射击槽位变量 $y_{k,m}$ 的数量由每个目标可接受的最大射击次数决定，拍照覆盖变量 u_p 固定为 18 个。由于所有约束均为线性约束，目标函数也通过 $y_{k,m}$ 和 u_p 完成线性化，模型可以直接交由 HiGHS 求解。

从数值稳定性看，本文没有使用极大或极小的 Big-M 常数。时间冲突和角度冲突均写成 $q_i + q_j \leq 1$ 或 $z_i + z_j \leq 1$，射击收益递减通过槽位变量线性化，拍照覆盖通过 u_p 的上下界连接完成。目标函数权重中最大系数为 $M_1 = 1000$，最小有效裕度系数为 $M_4 = 0.25$，二者相差四个数量级以内，且所有决策变量为 0–1 变量，因此求解器返回 gap=0 时可将方案解释为给定候选集和权重下的 MILP 最优解。

表 15　归一化任务效用与鲁棒性对比

方案	总事件	期望命中	拍照覆盖	综合效用	最小名义裕度	最坏裕度	平均 ρ_e	高风险事件
A 收益优先	25	8.0998	6	14.0998	0.0031	0.0000	0.8600	5
B 裕度优先	21	6.3993	5	11.3993	0.0337	0.0037	0.9762	0

表 15 说明，A/B 两个方案的差异不只是“多做几个任务”与“少做几个任务”，而是对边界事件采取了两种完全不同的处理口径。A 方案保留更多贴近约束边界的事件，因此综合效用最高，但最低名义裕度只有 0.0031，且仍含 5 个高风险事件；B 方案则主动放弃这类边界机会，把最低名义裕度抬到 0.0337、最坏裕度抬到 0.0037，平均场景通过率也提高到 0.9762。

因此，问题四的结果需要同时报告收益和风险口径，而不能只给出单个最优目标值。在同一候选集和同一模型结构下，收益优先会保留更多临界窗口，裕度优先则把这些窗口留作安全余量。前者适合任务完成数量优先的场景，后者适合执行失误代价更高的场景。

6.3.2 方案结果与 FOV 灵敏度

表 16 纯射击、纯拍照与联合方案比较

方案	射击事件	拍照事件	总事件	射击覆盖	拍照覆盖	期望命中	拍照观测	综合效用
纯射击	21	0	21	9	0	8.0998	0	8.0998
纯拍照	0	3	3	0	5	0.0000	5	5.0000
联合 A	21	4	25	9	6	8.0998	6	14.0998

表 16 中，联合 A 在保持 21 次射击事件和相同期望命中的同时，又插入 4 次拍照并覆盖 6 个拍照目标；相对纯拍照，它也保留了全部主要射击收益。该结果来自时间窗口的全局重排：射击与拍照并非始终互斥，真正稀缺的是少数局部时间段中的准备窗口资源。

换言之，联合优化之所以优于单任务方案，不是因为模型“更贪心”，而是因为它识别了哪些拍照事件可以嵌入射击间隙、哪些射击事件必须被保留。若只按单任务局部收益排序，早期高密度射击很容易挤占后续少数高价值拍照窗口，从而损失整体效用。

表 17 FOV 参数灵敏度

FOV/度	选择事件数	覆盖目标数	总观测次数	多目标同拍事件
30	4	4	4	0
45	3	5	5	2
60	3	6	7	3

FOV 增大后，单次拍照覆盖多个目标的能力增强，因此覆盖目标数和总观测次数同步上升；但这并不意味着 FOV 越大越好。因为 FOV 影响的是候选拍照事件的几何覆盖关系，而不是时间冲突结构本身，过大的视场角虽然会增加同拍机会，却也对应更强的成像与识别假设。本文主结果取 45°，本质上是在“覆盖潜力”与“感知假设克制性”之间取中间口径。

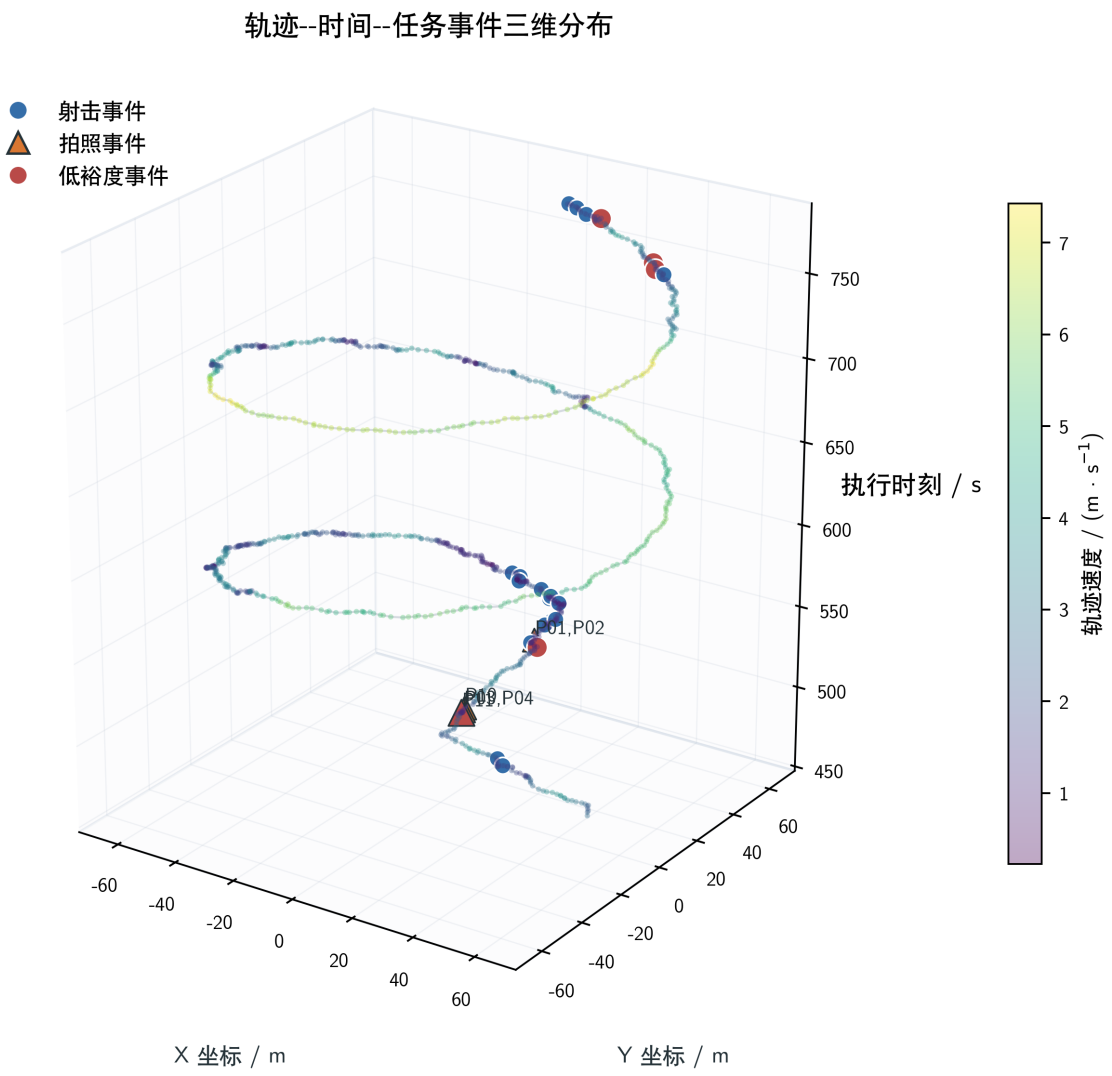

图 11　轨迹–时间–任务事件三维分布

图 11 将轨迹位置、时间推进和最终任务事件叠加在同一坐标系中，其中横、纵坐标为空间位置 X, Y，竖直方向为任务执行时刻。任务点集中在约 480–520 s 和 760 s 附近的两个时间簇，并非沿轨迹均匀分布；拍照事件主要嵌在前一时间簇的射击间隙中。图中单独标出的低裕度事件对应收益优先方案中被保留的边界窗口。

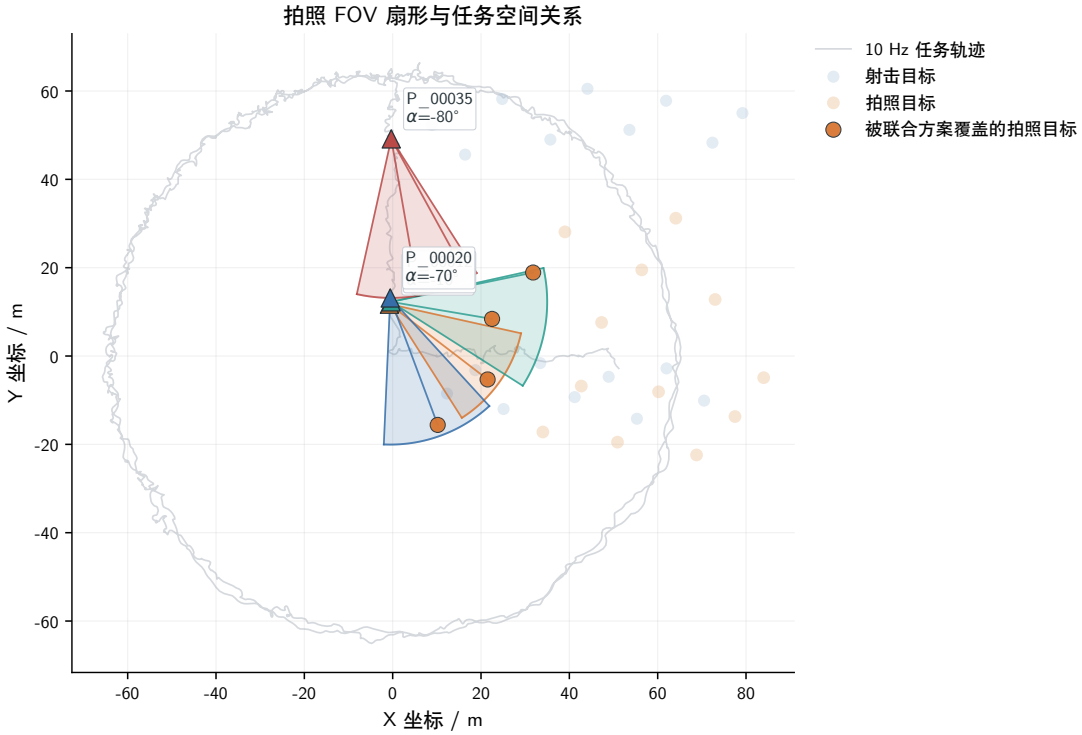

图 12　拍照 FOV 扇形与任务空间关系

图 12 直接揭示了拍照任务的几何本质：可行拍照点并不是沿轨迹均匀出现，而是集中在少数“轨迹经过且视线可同时扫过多个目标”的空间位置。P01/P02 与 P03/P04 的同拍覆盖，都来自单个 FOV 扇形与目标簇形成的几何重叠，而不是来自额外放宽时间或速度约束。也正因如此，拍照机会比射击机会稀缺得多。

这张图同时明确了“相机朝向可调假设”的边界。模型允许相机在候选时刻选取一个离散朝向角 α，但不允许在同一时刻连续扫掠多个方向；因此，多目标同拍来自真实的空间覆盖关系，而不是来自对相机能力的过度放宽。

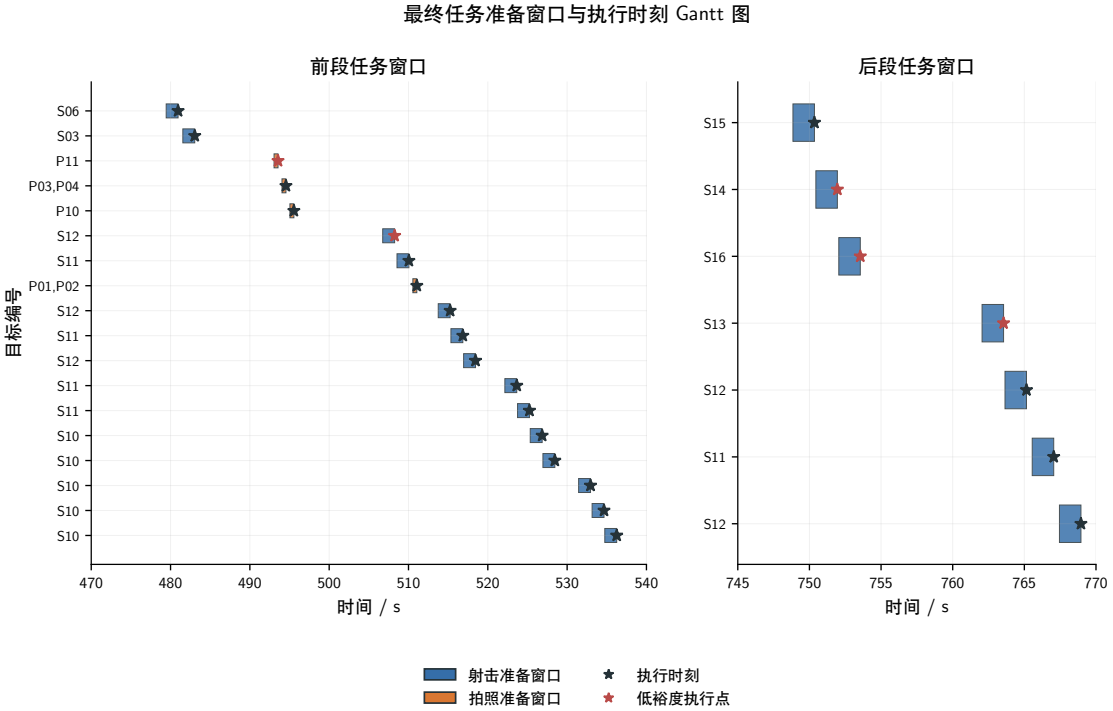

图 13　最终任务准备窗口与执行时刻 Gantt 图

图 13 显示，可执行任务并非均匀铺满全程，而是集中在两个局部时间簇内；这意味着问题四的实质矛盾不是“任务总量不够”，而是“高价值事件在少数时间段里相互挤压”。约 493–511 s 的几次拍照事件正是嵌入射击间隙后才被保留下来，这也是联合方案优于单任务方案的主要原因。若只按局部收益逐个选择，早期射击事件会迅速占满时间轴，后续多目标同拍机会就会整体消失。

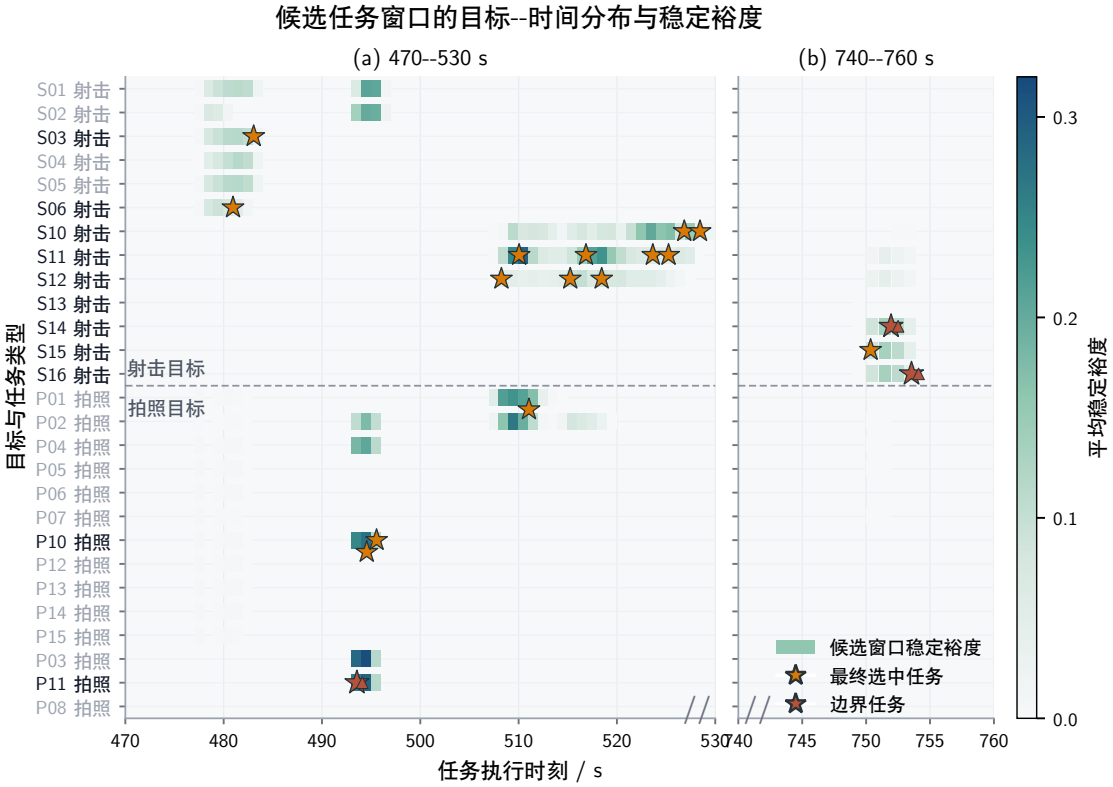

图 14　目标–时间候选事件可行性热力图

图 14 从目标–时间维度解释联合方案的结构。上半部分射击目标的候选窗口多于拍照目标，但大量窗口集中在相近时间段，因此会被准备窗口互斥约束压缩；下半部分拍照目标只有少数颜色明显的可行区域，说明 FOV 和运动学约束先决定“能不能看到”。最终选中事件落在两个时间簇内，机制上是因为这些时间簇同时满足距离、速度、加速度和覆盖条件；影响上看，MILP 的作用不是从全程均匀抽样，而是在少数局部窗口中重排冲突事件。

6.3.3　逐目标结果摘要

正文只保留覆盖结论，完整 18 个射击目标和 18 个拍照目标逐目标表见附录。主方案 A 覆盖 9 个射击目标，其中 S10、S11、S12 各安排 5 次射击，命中概率均达到 0.9999；S03、S06、S13、S14、S15、S16 各安排 1 次射击，命中概率为 0.85。未覆盖的射击目标主要原因是无可行窗口或被更高收益事件在时间冲突中挤出。拍照覆盖 P01、P02、P03、P04、P10、P11，其中 P01/P02 与 P03/P04 分别通过多目标同拍事件覆盖。

表 18　逐目标结果摘要

类型	分类	目标
射击	被选中	S03, S06, S10, S11, S12, S13, S14, S15, S16
射击	有窗口但未选	S01, S02, S04, S05
射击	无可行窗口	S07, S08, S09, S17, S18
拍照	被覆盖	P01, P02, P03, P04, P10, P11
拍照	无可行窗口	P05–P09, P12–P18

七、模型评价与灵敏度分析

7.1　模型适用场景

该模型适用于“轨迹已给定、目标已知、任务只能在轨迹上选择执行时刻”的移动机器人任务规划问题。定位融合部分要求两源观测来自同一机器人，且时间偏差可近似看成常数；若机器人路径发生改变或两源观测对象不同，则时间参数化配准不再适用。任务优化部分适用于射击、拍照、检测等离散事件调度问题，尤其适合存在准备时间、互斥窗口、多目标覆盖和收益递减的情形。若任务允许机器人主动改道，则还需要把路径规划与事件调度联合建模，固定轨迹 MILP 只能作为下层调度模块。

7.2　时间对齐稳定性分析

问题一中，时间参数化插值避免了路径回环或近垂直段带来的 $y = f(x)$ 多值问题。图 3 的 RMSE–Δ 曲线在最优点附近形成明确谷底，说明时间偏差识别由整体轨迹形状决定，而不是由少数采样点偶然匹配得到。附件 1 对齐后 RMSE 达到 10^{-8} m 量级，符合无噪声数据的数值精度预期。

7.3　固定偏差估计稳定性分析

问题二中，残差方差给出了随机噪声尺度，Q/R 灵敏度则给出了固定偏差估计的稳定性证据。三组过程噪声和观测噪声配置下，Δ 完全一致，b_x, b_y 变化小于 0.01 m，说明固定偏差并非参数调节造成的偶然结果。RTS 平滑对 RMSE 的改善有限，主要作用体现在平滑状态和运动学量输出上。

7.4　偏差结构模型泛化分析

问题三中，M3、M4 的训练误差低于 M1，但 5 折时间块平均 CV-RMSE 明显更高；分折结果还显示，M3 在前两折外推误差突出，M4 在第 2–4 折持续劣于 M1。按 Δ_{CV} 判据，复杂模型没有外推收益。附件 3 残差中确有结构性变化，但当前样本不足以证明这种变化可以稳定外推，因此后续只把它作为任务风险场景使用。

7.5 任务优化收益–风险折中

候选统计显示，大部分目标–时间组合被运动学、距离和 FOV 窗口剪除，最终 MILP 的计算难点主要来自 43744 条时间冲突边。Pareto 散点图中，从 A0 到 B2，综合效用从 14.0998 降到 10.5493，高风险事件数从 5 降到 0，场景通过率从 0.8600 提高到 0.9875。A 保留最高综合效用，B 则牺牲部分目标收益以换取无高风险事件。

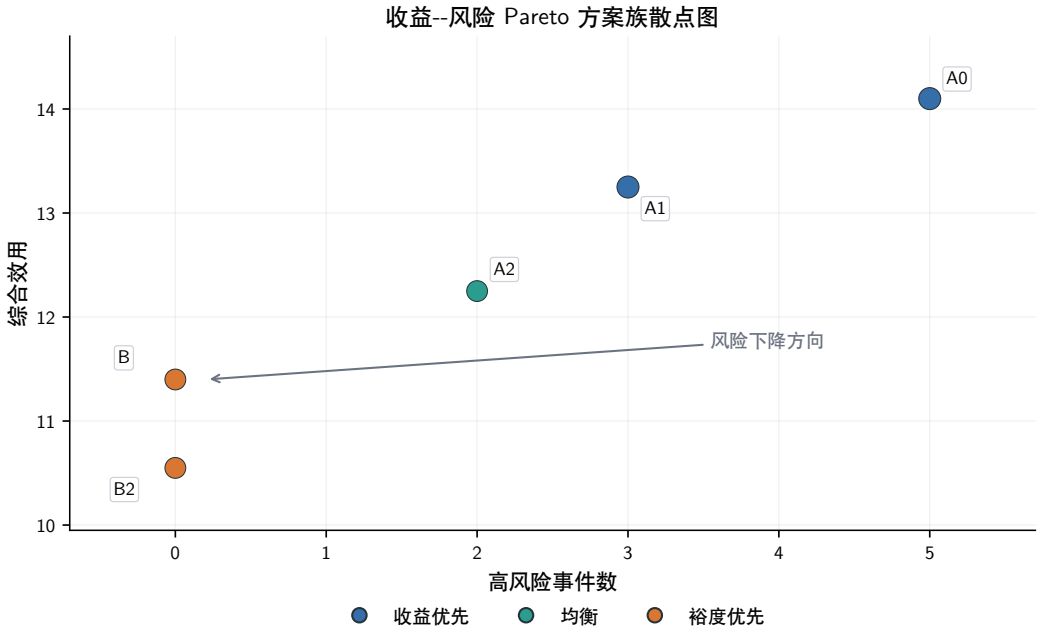

图 15　A/B 方案族收益–风险散点图

图 15 给出了从收益优先到裕度优先的一组 Pareto 方案。A 位于高收益端，保留最多任务覆盖和最高综合效用；B 位于低风险端，通过放弃部分边界事件将高风险事件数降为 0。中间散点则对应不同风险权重下的连续过渡，说明同一候选集可以通过参数调节表达不同风险偏好。

7.6 参数敏感性与可推广性

FOV 从 30° 增至 60° 时，拍照覆盖目标数从 4 增至 6，总观测次数从 4 增至 7，说明拍照结果对 FOV 具有实际敏感性。风险权重从 0.20 提高到 3.00 后，高风险事件被逐步替换或剔除，说明 MILP 框架可以通过权重和裕度阈值表达不同工程偏好。离散化步长敏感性表明，5 Hz 会夸大边界窗口的可行性，20 Hz 又会显著抬高冲突边规模并压缩最终综合效用，因此 10 Hz 是更平衡的事件枚举基准。另一方面，若不采用 RTS 状态输出和局部平滑差分，未平滑差分会把速度和加速度阈值整体推爆，从而使问题四候选池直接坍缩为 0；这也说明模型可推广的前提，不只是目标数可扩展，更是运动学输入口径必须保持稳定。模型可推广到更多目标或更长轨迹，因为候选事件生成、冲突边构造和 MILP 求解均不依赖 18 个目标或 9 行模板容量。

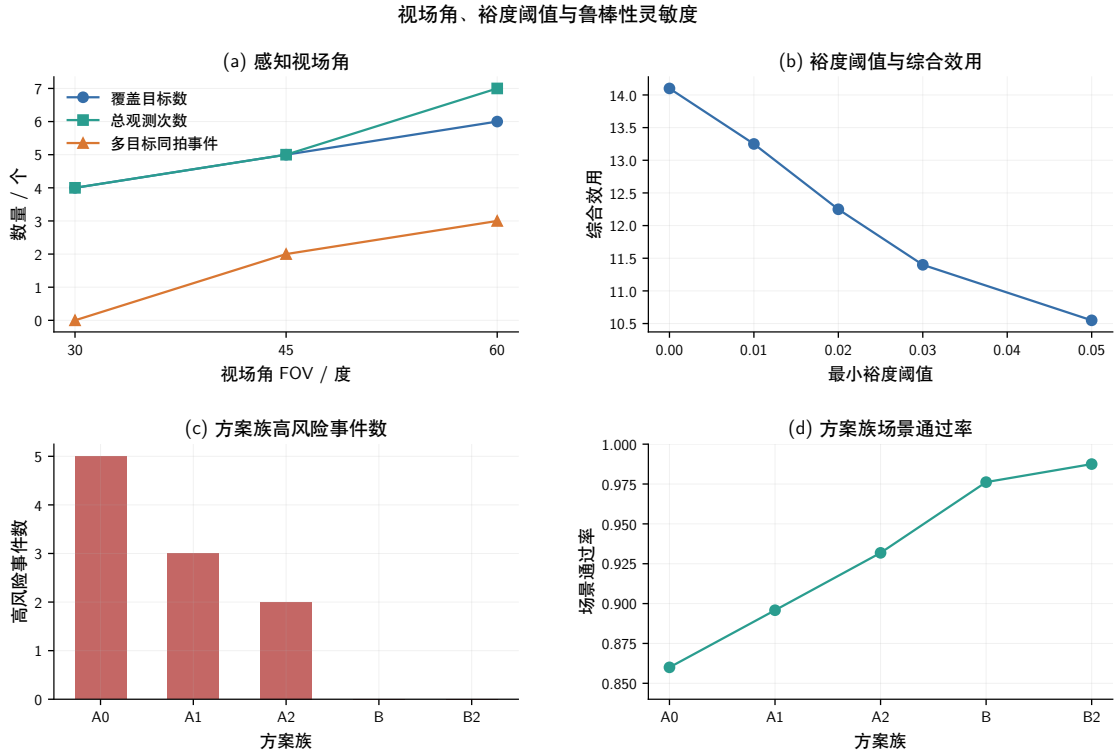

图 16　视场角、裕度阈值与鲁棒性灵敏度

图 16 说明，两类灵敏度参数作用在不同层面。FOV 改变的是“能看到什么”，因此主要影响拍照覆盖集合；裕度阈值和风险权重改变的是“敢不敢做”，因此主要影响边界事件的保留比例。把二者混为一谈，会误把感知能力变化解释成调度策略变化，或者反过来误把风险偏好变化解释成几何可见性变化。

表 19　10 Hz 离散化步长敏感性

Δt/s	频率/Hz	轨迹点数	射击候选	拍照候选	选择事件数	综合效用	时间冲突边
0.20	5	1500	676	70	27	15.7998	17101
0.10	10	3000	981	47	25	14.0998	43744
0.05	20	5999	1324	40	22	13.0989	102703

表 19 表明，离散化频率会改变候选事件空间，而不只是影响计算精度。5 Hz 时，时间采样较粗，若干本应分开的边界窗口被合并成更宽松的可行区，因此候选数偏少但综合效用偏高；20 Hz 时，更多短时波动和窗口重叠被显式展开，候选数与冲突边急剧增长，优化结果受到局部冲突压缩。10 Hz 在本题中兼顾了窗口边界保留和组合规模控制。

表 20　平滑运动学输入对问题四候选事件的影响

轨迹版本	运动学口径	速度 95 分位	加速度 95 分位	射击候选	拍照候选	联合候选	选择事件数	综合效用
未平滑差分	原始差分	8.2686	33.4420	0	0	0	0	0.0000
RTS 状态输出 + 局部平滑差分	81 点	6.5007	1.6998	981	47	1028	25	14.0998

表 20 把 RTS 的作用从“轨迹更平滑”具体推进到“任务能否成立”。这里的 81 点不是 RTS 本身的窗口，而是在 RTS 状态输出上进行局部平滑差分时采用的 8.1 s 运动学计算窗口，用于得到与 10 Hz 候选筛选一致的速度、加速度口径。若直接对含噪轨迹做差分，速度和加速度会被高频噪声成倍放大，所有候选事件都会在阈值筛选中被误杀；采用 RTS 状态输出和局部平滑差分后，候选池恢复为 1028 个事件，联合方案也才能形成 25 个有效任务。

7.7　模型优缺点

模型优点在于：时间参数化轨迹避免了路径几何多值问题；状态空间模型将固定偏差和随机误差分开，解释口径清楚；任务规划使用 MILP 明确给出变量、目标函数和冲突约束，主方案可声明为给定候选集和权重下的 MILP 最优解。其局限在于：附件 3 没有外部真值，残差结构目前只能作为扰动复核依据；FOV、目标识别质量和多目标同拍收益仍依赖题意假设；速度、加速度来自 RTS 状态输出和局部平滑差分，运动学计算窗口会影响边界任务；权重虽然做了尺度论证，但主备方案的最终选择仍需结合任务收益偏好与风险偏好。

模型的可解释性主要体现在三个层面：误差来源有对应参数，任务未被选择可追溯到冲突或不可行原因，风险方案由明确的权重和裕度阈值控制。附件 3 残差虽然存在结构性波动，但时间块 CV 没有支持稳定强校正，因此本文只将其用于扰动复核。

八、结论

本文将四个子问题统一为“轨迹可信度如何影响任务可靠性”的建模问题。附件 1 说明常值时间错位可以通过时间参数化轨迹稳定识别；附件 2 说明固定空间偏差可以与随机噪声分离，RTS 平滑的主要价值体现在运动学量而非位置 RMSE；附件 3 说明复杂残差模型虽能降低训练误差，但没有通过时间块外推检验，因此不应直接强校正。

在任务层面，事件级 MILP 给出了固定轨迹下的最优选择口径：A 方案获得最高综合效用 14.0998，B 方案则通过提高裕度阈值和风险权重把高风险事件降为 0。两类方案对应不同的执行偏好：A 侧重收益，B 侧重裕度。

综合四个问题，本文采用的处理口径是：稳定、可验证的误差进入名义轨迹；未能稳定外推的残差进入风险复核。这样既保留了固定偏差校正带来的轨迹一致性，也避免把局部残差纹理直接写成确定性任务窗口。

九、参考文献

[1] 乔少杰, 韩楠, 朱新文, 等. 基于卡尔曼滤波的动态轨迹预测算法 [J]. 电子学报, 2018, 46(2): 418–423.

[2] 王志. 移动机器人 RTK-GPS 循迹控制算法及其实现 [J]. 现代信息科技, 2025(14): 171–176.

[3] Kalman R E. A new approach to linear filtering and prediction problems[J]. Journal of Basic Engineering, 1960, 82(1): 35–45.

[4] Rauch H E, Tung F, Striebel C T. Maximum likelihood estimates of linear dynamic systems[J]. AIAA Journal, 1965, 3(8): 1445–1450.

[5] Hall D L, Llinas J. An introduction to multisensor data fusion[J]. Proceedings of the IEEE, 1997, 85(1): 6–23.

[6] Bar-Shalom Y, Li X R, Kirubarajan T. Estimation with Applications to Tracking and Navigation[M]. New York: Wiley, 2001.

[7] Efron B, Tibshirani R J. An Introduction to the Bootstrap[M]. New York: Chapman and Hall/CRC, 1993.

[8] Hastie T, Tibshirani R, Friedman J. The Elements of Statistical Learning[M]. 2nd ed. New York: Springer, 2009.

[9] Schölkopf B, Smola A J. Learning with Kernels[M]. Cambridge: MIT Press, 2002.

[10] Pinedo M L. Scheduling: Theory, Algorithms, and Systems[M]. 4th ed. New York: Springer, 2012.

[11] Wolsey L A. Integer Programming[M]. New York: Wiley, 1998.

[12] Nemhauser G L, Wolsey L A. Integer and Combinatorial Optimization[M]. New York: Wiley, 1988.

[13] Bertsimas D, Sim M. The price of robustness[J]. Operations Research, 2004, 52(1): 35–53.

A 支撑材料文件清单

按照竞赛论文提交要求，本文另附支撑材料压缩包 supporting_materials.zip。压缩包中不重复放入赛题提供的原始附件数据；复现程序运行时，需将赛题原始 附件 1.xlsx、附件 2.xlsx、附件 3.xlsx、附件 4.xlsx 以及结果模板 result.xlsx 放在代码根目录。除赛题原始数据外，支撑本文模型、结果和结论的源程序、主要中间结果、图表和填表结果均放入压缩包。

表 21 支撑材料压缩包目录说明

目录或文件	内容说明
README.md	支撑材料说明、运行环境和复现命令。
src/	建模所用完整 Python 源程序，共 13 个模块，包含数据读取、时间对齐、轨迹融合、卡尔曼滤波/RTS 平滑、残差结构检验、候选事件生成和 MILP/贪心回退优化。
scripts/	可运行复现入口，包括基础版、v2、v3 和最终版 v4 复现脚本。
paper_scripts/	论文图表整理脚本，用于把模型输出转换为正文中的高质量图表。
outputs/tables/	主要中间结果和最终结果表，共 103 个 CSV 文件，包括时间偏移、固定偏差、CV 误差、候选事件、冲突边、鲁棒性复核、A/B 方案和逐目标结果。
outputs/figures/	支撑正文与附录的主要图件，包括轨迹对齐、残差诊断、偏差模型比较、任务甘特图、风险折中曲线和候选热力图等。
outputs/trajectories/	10 Hz 融合轨迹、RTS 平滑轨迹和附件 3 任务优化轨迹等 CSV 文件。
outputs/logs/	模型选择、参数口径、鲁棒优化和迭代版本的文字记录。
outputs/result_filled*.xlsx	根据最终任务事件写入的结果工作簿文件。

B 核心代码说明

本文完整可运行源程序已放入支撑材料压缩包的 src/ 与 scripts/ 目录。最终复现命令为

```
python scripts/reproduce_model_results_v4.py.
```

该命令会重新生成附件 2 卡尔曼/RTS 偏差结果、附件 3 残差结构检验结果、附件 3 任务优化轨迹、联合射击/拍照任务方案和结果工作簿。本文没有使用 EXCEL、SPSS 等软件的交互命令进行建模计算；Excel 文件仅作为赛题数据输入和最终结果模板，全部建模计算由压缩包中的 Python 源程序完成。

表 22　核心源程序功能说明

源程序	功能
`src/alignment.py`	问题一核心：时间参数化轨迹、时间偏移搜索、重叠区间 RMSE 计算与 10 Hz 重采样。
`src/kalman_bias.py`	问题二核心：固定偏差初值、异步观测卡尔曼滤波、RTS 平滑与参数敏感性筛选。
`src/bias_structure.py`	问题三核心：残差特征构造、M0–M4 模型比较与时间块交叉验证。
`src/task_events.py`	问题四核心：射击/拍照候选事件生成、联合候选池构造与事件级 MILP 调度。
`scripts/reproduce_model_results_v4.py`	最终论文口径复现入口，串联问题二至问题四的完整计算流程。
其余辅助模块	数据读取、绘图诊断、结果回填和早期迭代脚本仍保留在支撑材料压缩包中，附录不再逐个展开。

2.1　核心 Python 源程序节选

附录只保留与正文模型直接对应的算法主干，以控制篇幅并突出建模核心。数据读取、图表渲染、结果回填、早期版本脚本和中间诊断函数不再重复贴入附录，完整可运行工程仍保留在支撑材料压缩包中。以下代码片段均通过 `\lstinputlisting` 从 `appendix_code/` 目录直接读入，和支撑材料中的对应源文件保持一致。

Listing 1: `src/alignment.py`

```python
from __future__ import annotations

from dataclasses import dataclass

import numpy as np
import pandas as pd
from scipy.interpolate import CubicSpline
from scipy.ndimage import uniform_filter1d
from scipy.optimize import minimize_scalar

from data_io import TIME_COL, X_COL, Y_COL


@dataclass
class AlignmentResult:
    delta: float
    bias_x: float
    bias_y: float
    rmse_before: float
    rmse_after: float
    overlap_start: float
    overlap_end: float


def _prepare(df: pd.DataFrame, smooth_window: int = 1) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
    clean = df[[TIME_COL, X_COL, Y_COL]].dropna().drop_duplicates(TIME_COL).sort_values(TIME_COL)
    t = clean[TIME_COL].to_numpy(dtype=float)
    x = clean[X_COL].to_numpy(dtype=float)
    y = clean[Y_COL].to_numpy(dtype=float)
    if smooth_window > 1:
        x = uniform_filter1d(x, size=smooth_window, mode="nearest")
        y = uniform_filter1d(y, size=smooth_window, mode="nearest")
    return t, x, y


def _splines(t: np.ndarray, x: np.ndarray, y: np.ndarray) -> tuple[CubicSpline, CubicSpline]:
    return CubicSpline(t, x), CubicSpline(t, y)


def _overlap(t1: np.ndarray, t2: np.ndarray, delta: float) -> tuple[float, float]:
    return max(t1[0], t2[0] + delta), min(t1[-1], t2[-1] + delta)
```

```python


def _sample_grid(start: float, end: float, step: float = 0.1) -> np.ndarray:
    if end <= start:
        return np.array([])
    n = int(np.floor((end - start) / step)) + 1
    return start + np.arange(n) * step


def rmse_for_delta(
    t1: np.ndarray,
    x1: np.ndarray,
    y1: np.ndarray,
    t2: np.ndarray,
    x2: np.ndarray,
    y2: np.ndarray,
    delta: float,
    estimate_bias: bool,
) -> float:
    start, end = _overlap(t1, t2, delta)
    grid = _sample_grid(start, end, 0.2)
    if len(grid) < 10:
        return float("inf")
    s1x, s1y = _splines(t1, x1, y1)
    s2x, s2y = _splines(t2, x2, y2)
    dx = s2x(grid - delta) - s1x(grid)
    dy = s2y(grid - delta) - s1y(grid)
    if estimate_bias:
        dx = dx - np.nanmedian(dx)
        dy = dy - np.nanmedian(dy)
    return float(np.sqrt(np.nanmean(dx * dx + dy * dy)))


def align_pair(
    df1: pd.DataFrame,
    df2: pd.DataFrame,
    estimate_bias: bool,
    smooth_window: int = 1,
    search_margin: float = 5.0,
) -> AlignmentResult:
    t1, x1, y1 = _prepare(df1, smooth_window=smooth_window)
    t2, x2, y2 = _prepare(df2, smooth_window=smooth_window)

    start_delta = t1[0] - t2[0]
    end_delta = t1[-1] - t2[-1]
    center = float((start_delta + end_delta) / 2.0)
    half_width = abs(start_delta - end_delta) / 2.0 + max(search_margin, 120.0)
    lo, hi = center - half_width, center + half_width

    objective = lambda d: rmse_for_delta(t1, x1, y1, t2, x2, y2, float(d), estimate_bias)
    coarse = np.linspace(lo, hi, 201)
    coarse_scores = np.array([objective(d) for d in coarse])
    best = float(coarse[int(np.nanargmin(coarse_scores))])
    bracket_lo = max(lo, best - max(2.0, half_width / 20))
    bracket_hi = min(hi, best + max(2.0, half_width / 20))
    opt = minimize_scalar(objective, bounds=(bracket_lo, bracket_hi), method="bounded", options={"xatol": 1e-4})
    delta = float(opt.x)

    start, end = _overlap(t1, t2, delta)
    grid = _sample_grid(start, end, 0.1)
    s1x, s1y = _splines(t1, x1, y1)
    s2x, s2y = _splines(t2, x2, y2)
    diff_x = s2x(grid - delta) - s1x(grid)
    diff_y = s2y(grid - delta) - s1y(grid)
    bias_x = float(np.nanmedian(diff_x)) if estimate_bias else 0.0
    bias_y = float(np.nanmedian(diff_y)) if estimate_bias else 0.0
    rmse_before = float(np.sqrt(np.nanmean(diff_x * diff_x + diff_y * diff_y)))
    adj_x = diff_x - bias_x
    adj_y = diff_y - bias_y
    rmse_after = float(np.sqrt(np.nanmean(adj_x * adj_x + adj_y * adj_y)))

    return AlignmentResult(delta, bias_x, bias_y, rmse_before, rmse_after, float(start), float(end))


def resample_aligned(
    df1: pd.DataFrame,
    df2: pd.DataFrame,
    result: AlignmentResult,
```

```
    smooth_window: int = 1,
) -> pd.DataFrame:
    t1, x1, y1 = _prepare(df1, smooth_window=smooth_window)
    t2, x2, y2 = _prepare(df2, smooth_window=smooth_window)
    grid = _sample_grid(result.overlap_start, result.overlap_end, 0.1)
    s1x, s1y = _splines(t1, x1, y1)
    s2x, s2y = _splines(t2, x2, y2)
    x1g, y1g = s1x(grid), s1y(grid)
    x2g = s2x(grid - result.delta) - result.bias_x
    y2g = s2y(grid - result.delta) - result.bias_y
    return pd.DataFrame(
        {
            TIME_COL: np.round(grid, 3),
            "x1_aligned": x1g,
            "y1_aligned": y1g,
            "x2_aligned_corrected": x2g,
            "y2_aligned_corrected": y2g,
            X_COL: (x1g + x2g) / 2.0,
            Y_COL: (y1g + y2g) / 2.0,
        }
    )
```

Listing 2: `src/kalman_bias.py`（固定偏差识别、卡尔曼滤波与 RTS 平滑核心段）

```
from __future__ import annotations

from dataclasses import dataclass
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from scipy.interpolate import CubicSpline

from alignment import AlignmentResult, align_pair, resample_aligned
from data_io import TIME_COL, X_COL, Y_COL
from plotting import _setup_font


@dataclass
class KalmanConfig:
    """Noise parameters for the multi-rate Kalman bias model."""

    name: str
    q_pos: float
    q_vel: float
    q_bias: float
    r1: float
    r2: float


def _clean(df: pd.DataFrame) -> pd.DataFrame:
    return df[[TIME_COL, X_COL, Y_COL]].dropna().drop_duplicates(TIME_COL).sort_values(TIME_COL).reset_index(drop=True)


def _event_table(df1: pd.DataFrame, df2: pd.DataFrame, delta: float) -> pd.DataFrame:
    """Merge asynchronous source-1 and source-2 observations after shifting source 2 by delta."""

    a = _clean(df1).copy()
    b = _clean(df2).copy()
    a["source"] = 1
    b["source"] = 2
    b[TIME_COL] = b[TIME_COL] + delta
    start = max(float(a[TIME_COL].min()), float(b[TIME_COL].min()))
    end = min(float(a[TIME_COL].max()), float(b[TIME_COL].max()))
    obs = pd.concat([a, b], ignore_index=True)
    obs = obs[(obs[TIME_COL] >= start) & (obs[TIME_COL] <= end)].copy()
    obs = obs.sort_values([TIME_COL, "source"]).reset_index(drop=True)
    return obs


def _transition(dt: float) -> np.ndarray:
    f = np.eye(6)
    f[0, 2] = dt
    f[1, 3] = dt
    return f
```

```python


def _process_noise(dt: float, cfg: KalmanConfig) -> np.ndarray:
    q = np.diag([cfg.q_pos, cfg.q_pos, cfg.q_vel, cfg.q_vel, cfg.q_bias, cfg.q_bias])
    return q * max(dt, 1e-3)


def _obs_model(source: int) -> np.ndarray:
    h = np.zeros((2, 6))
    h[0, 0] = 1.0
    h[1, 1] = 1.0
    if source == 2:
        h[0, 4] = 1.0
        h[1, 5] = 1.0
    return h


def _initial_state(obs: pd.DataFrame, baseline: AlignmentResult) -> tuple[np.ndarray, np.ndarray]:
    first = obs.iloc[0]
    x0 = np.zeros(6)
    if int(first["source"]) == 2:
        x0[0] = float(first[X_COL]) - baseline.bias_x
        x0[1] = float(first[Y_COL]) - baseline.bias_y
    else:
        x0[0] = float(first[X_COL])
        x0[1] = float(first[Y_COL])
    x0[4] = baseline.bias_x
    x0[5] = baseline.bias_y
    p0 = np.diag([10.0, 10.0, 5.0, 5.0, 5.0, 5.0])
    return x0, p0


def kalman_filter(df1: pd.DataFrame, df2: pd.DataFrame, delta: float, baseline: AlignmentResult, cfg: KalmanConfig) -> dict[str, object]:
    """Run a multi-rate Kalman filter with fixed source-2 bias states."""

    obs = _event_table(df1, df2, delta)
    if len(obs) < 5:
        raise ValueError("not enough overlapping observations for Kalman filtering")
    x, p = _initial_state(obs, baseline)
    xs_f, ps_f, xs_pred, ps_pred, fs = [], [], [], [], []
    nll = 0.0
    prev_t = float(obs.iloc[0][TIME_COL])
    eye = np.eye(6)
    for _, row in obs.iterrows():
        t = float(row[TIME_COL])
        dt = max(0.0, t - prev_t)
        f = _transition(dt)
        q = _process_noise(dt, cfg)
        x_pred = f @ x
        p_pred = f @ p @ f.T + q
        h = _obs_model(int(row["source"]))
        r_scale = cfg.r1 if int(row["source"]) == 1 else cfg.r2
        r = np.eye(2) * max(r_scale, 1e-6)
        z = np.array([float(row[X_COL]), float(row[Y_COL])])
        nu = z - h @ x_pred
        s = h @ p_pred @ h.T + r
        s_inv = np.linalg.pinv(s)
        k = p_pred @ h.T @ s_inv
        x = x_pred + k @ nu
        p = (eye - k @ h) @ p_pred @ (eye - k @ h).T + k @ r @ k.T
        sign, logdet = np.linalg.slogdet(s)
        nll += float(nu.T @ s_inv @ nu + (logdet if sign > 0 else 0.0))
        xs_pred.append(x_pred)
        ps_pred.append(p_pred)
        xs_f.append(x.copy())
        ps_f.append(p.copy())
        fs.append(f)
        prev_t = t
    return {
        "obs": obs,
        "x_filter": np.vstack(xs_f),
        "p_filter": np.stack(ps_f),
        "x_pred": np.vstack(xs_pred),
        "p_pred": np.stack(ps_pred),
        "f": fs,
        "nll": nll,
    }

```

```python

def rts_smoother(filtered: dict[str, object]) -> np.ndarray:
    """Apply Rauch-Tung-Striebel smoothing to filtered states."""

    xf = np.asarray(filtered["x_filter"])
    pf = np.asarray(filtered["p_filter"])
    xp = np.asarray(filtered["x_pred"])
    pp = np.asarray(filtered["p_pred"])
    fs = filtered["f"]
    xs = xf.copy()
    ps = pf.copy()
    for k in range(len(xf) - 2, -1, -1):
        c = pf[k] @ fs[k + 1].T @ np.linalg.pinv(pp[k + 1])
        xs[k] = xf[k] + c @ (xs[k + 1] - xp[k + 1])
        ps[k] = pf[k] + c @ (ps[k + 1] - pp[k + 1]) @ c.T
    return xs


def _state_to_10hz(obs: pd.DataFrame, xs: np.ndarray) -> pd.DataFrame:
    t = obs[TIME_COL].to_numpy(float)
    order = np.argsort(t)
    t = t[order]
    xs = xs[order]
    _, unique_idx = np.unique(t, return_index=True)
    unique_idx = np.sort(unique_idx)
    t = t[unique_idx]
    xs = xs[unique_idx]
    grid = np.round(np.arange(t[0], t[-1] + 1e-9, 0.1), 3)
    out = {TIME_COL: grid}
    for col, idx in [("X坐标(m)", 0), ("Y坐标(m)", 1), ("vx", 2), ("vy", 3), ("bias_x", 4), ("bias_y", 5)]:
        out[col] = np.interp(grid, t, xs[:, idx])
    return pd.DataFrame(out)


def _rmse_after(df1: pd.DataFrame, df2: pd.DataFrame, delta: float, bx: float, by: float, smooth_window: int = 5) -> tuple[float, float, float, float]:
    a = _clean(df1)
    b = _clean(df2)
    if smooth_window > 1:
        a = a.copy()
        b = b.copy()
        a[X_COL] = pd.Series(a[X_COL]).rolling(smooth_window, center=True, min_periods=1).mean()
        a[Y_COL] = pd.Series(a[Y_COL]).rolling(smooth_window, center=True, min_periods=1).mean()
        b[X_COL] = pd.Series(b[X_COL]).rolling(smooth_window, center=True, min_periods=1).mean()
        b[Y_COL] = pd.Series(b[Y_COL]).rolling(smooth_window, center=True, min_periods=1).mean()
    start = max(float(a[TIME_COL].min()), float(b[TIME_COL].min() + delta))
    end = min(float(a[TIME_COL].max()), float(b[TIME_COL].max() + delta))
    grid = np.arange(start, end + 1e-9, 0.1)
    s1x, s1y = CubicSpline(a[TIME_COL], a[X_COL]), CubicSpline(a[TIME_COL], a[Y_COL])
    s2x, s2y = CubicSpline(b[TIME_COL], b[X_COL]), CubicSpline(b[TIME_COL], b[Y_COL])
    before_x = s2x(grid - delta) - s1x(grid)
    before_y = s2y(grid - delta) - s1y(grid)
    after_x = before_x - bx
    after_y = before_y - by
    return (
        float(np.sqrt(np.mean(before_x**2 + before_y**2))),
        float(np.sqrt(np.mean(after_x**2 + after_y**2))),
        float(np.var(after_x)),
        float(np.var(after_y)),
    )


def run_kalman_bias_attachment2(df1: pd.DataFrame, df2: pd.DataFrame, out_dir: Path) -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame]:
    """Estimate fixed source-2 bias after noise separation for attachment 2."""

    tables = out_dir / "tables"
    figs = out_dir / "figures"
    traj_dir = out_dir / "trajectories"
    for p in [tables, figs, traj_dir]:
        p.mkdir(parents=True, exist_ok=True)

    baseline = align_pair(df1, df2, estimate_bias=True, smooth_window=5)
    rmse_before, rmse_after, var_x, var_y = _rmse_after(df1, df2, baseline.delta, baseline.bias_x, baseline.bias_y)
    r_base = max(0.05, (var_x + var_y) / 2.0)
    configs = [
        KalmanConfig("low_process", 0.002, 0.010, 1e-7, r_base, r_base),
        KalmanConfig("balanced", 0.010, 0.050, 1e-6, r_base, r_base),
        KalmanConfig("smooth_bias", 0.030, 0.100, 1e-8, r_base * 1.2, r_base * 1.2),
    ]
```

```
    sens_rows = []
    best = None
    for cfg in configs:
        deltas = np.linspace(baseline.delta - 1.0, baseline.delta + 1.0, 21)
        best_for_cfg = None
        for delta in deltas:
            filtered = kalman_filter(df1, df2, float(delta), baseline, cfg)
            xf = np.asarray(filtered["x_filter"])
            bx, by = float(np.median(xf[:, 4])), float(np.median(xf[:, 5]))
            _before, after, vx, vy = _rmse_after(df1, df2, float(delta), bx, by)
            score = float(filtered["nll"])
            if best_for_cfg is None or score < best_for_cfg["nll"]:
                best_for_cfg = {
                    "config": cfg,
                    "Delta": float(delta),
                    "bx": bx,
                    "by": by,
                    "rmse_after": after,
                    "residual_var_x": vx,
                    "residual_var_y": vy,
                    "nll": score,
                    "filtered": filtered,
                }
        assert best_for_cfg is not None
        sens_rows.append(
            {
                "config_name": cfg.name,
                "q_pos": cfg.q_pos,
                "q_vel": cfg.q_vel,
                "q_bias": cfg.q_bias,
                "r1_x": cfg.r1,
                "r1_y": cfg.r1,
                "r2_x": cfg.r2,
                "r2_y": cfg.r2,
                "Delta": best_for_cfg["Delta"],
                "bx": best_for_cfg["bx"],
                "by": best_for_cfg["by"],
                "rmse_after": best_for_cfg["rmse_after"],
                "residual_var_x": best_for_cfg["residual_var_x"],
                "residual_var_y": best_for_cfg["residual_var_y"],
                "nll": best_for_cfg["nll"],
            }
        )
        if best is None or best_for_cfg["nll"] < best["nll"]:
            best = best_for_cfg
    assert best is not None
    xs = rts_smoother(best["filtered"])
    smoothed = _state_to_10hz(best["filtered"]["obs"], xs)
    smoothed.to_csv(traj_dir / "attachment2_kalman_rts_10hz.csv", index=False, encoding="utf-8-sig")
    bx_rts, by_rts = float(np.median(xs[:, 4])), float(np.median(xs[:, 5]))
    _before, after_rts, vx_rts, vy_rts = _rmse_after(df1, df2, best["Delta"], bx_rts, by_rts)

    summary = pd.DataFrame(
        [
            {
                "method": "RobustMedian",
                "Delta": baseline.delta,
                "bx": baseline.bias_x,
                "by": baseline.bias_y,
                "rmse_before": rmse_before,
                "rmse_after": rmse_after,
                "residual_var_x": var_x,
                "residual_var_y": var_y,
                "nll": np.nan,
                "notes": "baseline robust median after coarse-to-fine RMSE alignment",
            },
            {
                "method": "KalmanFilter",
                "Delta": best["Delta"],
                "bx": best["bx"],
                "by": best["by"],
                "rmse_before": rmse_before,
                "rmse_after": best["rmse_after"],
                "residual_var_x": best["residual_var_x"],
                "residual_var_y": best["residual_var_y"],
                "nll": best["nll"],
                "notes": f"best sensitivity config={best['config'].name}",
            },
```

```
            {
                "method": "KalmanRTS",
                "Delta": best["Delta"],
                "bx": bx_rts,
                "by": by_rts,
                "rmse_before": rmse_before,
                "rmse_after": after_rts,
                "residual_var_x": vx_rts,
                "residual_var_y": vy_rts,
                "nll": best["nll"],
                "notes": "RTS smoothed states using all observations",
            },
        ]
    )
    sensitivity = pd.DataFrame(sens_rows)
    summary.to_csv(tables / "kalman_bias_attachment2_summary.csv", index=False, encoding="utf-8-sig")
    sensitivity.to_csv(tables / "kalman_bias_sensitivity.csv", index=False, encoding="utf-8-sig")
```

Listing 3: `src/bias_structure.py`（残差特征构造与时间块交叉验证核心段）

```
from __future__ import annotations

from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from scipy.interpolate import CubicSpline
from scipy.ndimage import uniform_filter1d
from sklearn.kernel_ridge import KernelRidge
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.metrics import mean_squared_error
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

from alignment import align_pair
from data_io import TIME_COL, X_COL, Y_COL
from kinematics import add_kinematics
from plotting import _setup_font


def _clean(df: pd.DataFrame, smooth_window: int = 1) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
    part = df[[TIME_COL, X_COL, Y_COL]].dropna().drop_duplicates(TIME_COL).sort_values(TIME_COL)
    t = part[TIME_COL].to_numpy(float)
    x = part[X_COL].to_numpy(float)
    y = part[Y_COL].to_numpy(float)
    if smooth_window > 1:
        x = uniform_filter1d(x, smooth_window, mode="nearest")
        y = uniform_filter1d(y, smooth_window, mode="nearest")
    return t, x, y


def _residual_frame(df1: pd.DataFrame, df2: pd.DataFrame, delta: float, smooth_window: int = 7) -> pd.DataFrame:
    t1, x1, y1 = _clean(df1, smooth_window)
    t2, x2, y2 = _clean(df2, smooth_window)
    start = max(t1[0], t2[0] + delta)
    end = min(t1[-1], t2[-1] + delta)
    grid = np.arange(start, end + 1e-9, 0.2)
    s1x, s1y = CubicSpline(t1, x1), CubicSpline(t1, y1)
    s2x, s2y = CubicSpline(t2, x2), CubicSpline(t2, y2)
    x1g, y1g = s1x(grid), s1y(grid)
    x2g, y2g = s2x(grid - delta), s2y(grid - delta)
    fused = pd.DataFrame({TIME_COL: grid, X_COL: (x1g + x2g) / 2, Y_COL: (y1g + y2g) / 2})
    fused = add_kinematics(fused, smooth_window=41)
    theta = np.arctan2(fused["vy"].to_numpy(float), fused["vx"].to_numpy(float))
    speed = fused["speed"].to_numpy(float)
    dt = float(np.median(np.diff(grid)))
    heading_rate = np.gradient(theta, dt)
    curvature = np.abs(heading_rate) / np.maximum(speed, 1e-6)
    return pd.DataFrame(
        {
            TIME_COL: grid,
            "residual_x": x2g - x1g,
            "residual_y": y2g - y1g,
            "t_norm": (grid - grid.min()) / max(grid.max() - grid.min(), 1e-9),
            "x": fused[X_COL],
```

```
            "y": fused[Y_COL],
            "v": speed,
            "a": fused["acceleration"],
            "theta": theta,
            "cos_theta": np.cos(theta),
            "sin_theta": np.sin(theta),
            "curvature": curvature,
        }
    )


def _rmse(y_true: np.ndarray, y_pred: np.ndarray) -> float:
    return float(np.sqrt(mean_squared_error(y_true, y_pred)))


def _blocked_folds(n: int, k: int = 5) -> list[np.ndarray]:
    k = min(k, max(3, n // 30))
    edges = np.linspace(0, n, k + 1, dtype=int)
    return [np.arange(edges[i], edges[i + 1]) for i in range(k) if edges[i + 1] > edges[i]]


def _fit_predict(model_name: str, x_train: np.ndarray, y_train: np.ndarray, x_test: np.ndarray) -> tuple[np.ndarray, object, int]:
    if model_name == "M0":
        return np.zeros((len(x_test), 2)), None, 0
    if model_name == "M1":
        mean = y_train.mean(axis=0)
        return np.repeat(mean[None, :], len(x_test), axis=0), mean, 2
    if model_name == "M2":
        reg = LinearRegression().fit(x_train[:, [0]], y_train)
        return reg.predict(x_test[:, [0]]), reg, 4
    if model_name == "M3":
        reg = make_pipeline(StandardScaler(), Ridge(alpha=1.0)).fit(x_train, y_train)
        return reg.predict(x_test), reg, 16
    if model_name == "M4":
        reg = make_pipeline(StandardScaler(), KernelRidge(alpha=2.0, kernel="rbf", gamma=0.35)).fit(x_train, y_train)
        return reg.predict(x_test), reg, len(x_train)
    raise ValueError(model_name)


def run_attachment3_bias_structure(df1: pd.DataFrame, df2: pd.DataFrame, out_dir: Path) -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame]:
    """Compare fixed, time-varying, state-related, and nonlinear residual-bias models."""

    tables = out_dir / "tables"
    figs = out_dir / "figures"
    tables.mkdir(parents=True, exist_ok=True)
    figs.mkdir(parents=True, exist_ok=True)
    baseline = align_pair(df1, df2, estimate_bias=True, smooth_window=7)
    data = _residual_frame(df1, df2, baseline.delta, smooth_window=7)
    features = ["t_norm", "x", "y", "v", "a", "cos_theta", "sin_theta", "curvature"]
    x = data[features].to_numpy(float)
    y = data[["residual_x", "residual_y"]].to_numpy(float)
    folds = _blocked_folds(len(data), 5)
    descriptions = {
        "M0": "no systematic bias, residual prediction is zero",
        "M1": "constant translation bias",
        "M2": "linear time-varying bias",
        "M3": "state-related linear Ridge bias",
        "M4": "nonlinear RBF kernel-ridge bias with blocked CV",
    }
    rows = []
    full_models = {}
    m0_cv = None
    m1_cv = None
    for name in ["M0", "M1", "M2", "M3", "M4"]:
        pred_train, model, n_params = _fit_predict(name, x, y, x)
        train_rmse = _rmse(y, pred_train)
        cv_preds = np.zeros_like(y)
        for test_idx in folds:
            train_mask = np.ones(len(data), dtype=bool)
            train_mask[test_idx] = False
            pred, _, _ = _fit_predict(name, x[train_mask], y[train_mask], x[test_idx])
            cv_preds[test_idx] = pred
        cv_rmse = _rmse(y, cv_preds)
        cv_rmse_x = float(np.sqrt(mean_squared_error(y[:, 0], cv_preds[:, 0])))
        cv_rmse_y = float(np.sqrt(mean_squared_error(y[:, 1], cv_preds[:, 1])))
        if name == "M0":
            m0_cv = cv_rmse
        if name == "M1":
```

```
            m1_cv = cv_rmse
        imp0 = 0.0 if not m0_cv else (m0_cv - cv_rmse) / m0_cv
        imp1 = 0.0 if not m1_cv else (m1_cv - cv_rmse) / m1_cv
        if name == "M4" and train_rmse < cv_rmse * 0.75:
            conclusion = "nonlinear training gain, CV suggests overfit risk"
        elif name in {"M2", "M3", "M4"} and imp1 >= 0.05:
            conclusion = "structured non-fixed systematic bias supported"
        elif name == "M1" and imp0 >= 0.05:
            conclusion = "constant translation explains part of residual"
        else:
            conclusion = "not primary explanatory model"
        rows.append(
            {
                "model_name": name,
                "description": descriptions[name],
                "n_params": n_params,
                "train_rmse": train_rmse,
                "cv_rmse": cv_rmse,
                "cv_rmse_x": cv_rmse_x,
                "cv_rmse_y": cv_rmse_y,
                "improvement_vs_M0": imp0,
                "improvement_vs_M1": imp1,
                "conclusion": conclusion,
            }
        )
        full_models[name] = model
    comparison = pd.DataFrame(rows)
    comparison.to_csv(tables / "attachment3_bias_structure_comparison.csv", index=False, encoding="utf-8-sig")

    coeff_rows = []
    model_m3 = full_models["M3"]
    if model_m3 is not None:
        ridge = model_m3.named_steps["ridge"]
        for axis, coefs in zip(["x", "y"], ridge.coef_):
            for feature, coef in zip(features, coefs):
                coeff_rows.append({"model_name": "M3", "axis": axis, "feature": feature, "coefficient": float(coef)})
    coeffs = pd.DataFrame(coeff_rows)
    coeffs.to_csv(tables / "attachment3_bias_feature_coefficients.csv", index=False, encoding="utf-8-sig")
    data.to_csv(tables / "attachment3_residual_feature_frame.csv", index=False, encoding="utf-8-sig")
    _plot_bias_structure(data, comparison, figs)
    return comparison, coeffs, data
```

Listing 4: `src/task_events.py`（射击候选事件生成）

```
from __future__ import annotations

import math
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from scipy.optimize import Bounds, LinearConstraint, milp

from data_io import TIME_COL, X_COL, Y_COL
from plotting import _setup_font


RNG_SEED = 42
SHOT_HIT_PROB = 0.85


def circular_angle_diff(a: float, b: float) -> float:
    """Smallest circular angle difference in degrees."""

    raw = abs(float(a) - float(b)) % 360.0
    return min(raw, 360.0 - raw)


def _intervals_from_times(times: list[float]) -> str:
    if not times:
        return ""
    vals = sorted(times)
    runs = []
    start = prev = vals[0]
    for t in vals[1:]:
```

```
        if t - prev <= 0.25:
            prev = t
        else:
            runs.append(f"{start:.1f}-{prev:.1f}")
            start = prev = t
    runs.append(f"{start:.1f}-{prev:.1f}")
    return ";".join(runs[:8])


def _margin(dwin: np.ndarray, speed: np.ndarray, acc: np.ndarray, dmin: float, dmax: float, vmax: float, amax: float) -> float:
    vals = [
        np.min((dwin - dmin) / (dmax - dmin)),
        np.min((dmax - dwin) / (dmax - dmin)),
        np.min((vmax - speed) / vmax),
        np.min((amax - acc) / amax),
    ]
    return float(np.min(vals))


def _conflicts(a_start: float, a_end: float, b_start: float, b_end: float) -> bool:
    return not (a_end < b_start or b_end < a_start)


def _risk_penalty(margin: float) -> float:
    return 1.0 if margin < 0.03 else 0.35 if margin < 0.08 else 0.0


def split_targets(targets: pd.DataFrame) -> tuple[pd.DataFrame, pd.DataFrame]:
    """Return shooting and photo target tables."""

    return (
        targets[targets["任务"] == "射击"].copy().reset_index(drop=True),
        targets[targets["任务"] == "拍照"].copy().reset_index(drop=True),
    )


def generate_shooting_events(traj: pd.DataFrame, shooting_targets: pd.DataFrame, stride: int = 1) -> pd.DataFrame:
    """Enumerate all feasible shooting events on a 10 Hz trajectory."""

    rows = []
    xy = traj[[X_COL, Y_COL]].to_numpy(float)
    speed = traj["speed"].to_numpy(float)
    acc = traj["acceleration"].to_numpy(float)
    times = traj[TIME_COL].to_numpy(float)
    for _, target in shooting_targets.iterrows():
        tx, ty = float(target[X_COL]), float(target[Y_COL])
        dist = np.sqrt((xy[:, 0] - tx) ** 2 + (xy[:, 1] - ty) ** 2)
        feasible_times = []
        for i in range(15, len(traj), stride):
            dwin = dist[i - 15 : i + 1]
            swin = speed[i - 15 : i + 1]
            awin = acc[i - 15 : i + 1]
            if np.all((dwin >= 5.0) & (dwin <= 30.0)) and np.all(swin <= 2.0) and np.all(awin <= 1.5):
                feasible_times.append(float(times[i]))
                m = _margin(dwin, swin, awin, 5.0, 30.0, 2.0, 1.5)
                rows.append(
                    {
                        "event_id": f"S_{len(rows):05d}",
                        "event_type": "shooting",
                        "target_id": str(target["编号"]),
                        "start_time": float(times[i - 15]),
                        "execute_time": float(times[i]),
                        "distance": float(dist[i]),
                        "speed": float(speed[i]),
                        "acceleration": float(acc[i]),
                        "margin": m,
                    }
                )
    return pd.DataFrame(rows)
```

Listing 5: `src/task_events.py`（拍照候选事件生成）

```
def generate_photo_events(traj: pd.DataFrame, photo_targets: pd.DataFrame, fov_degree: float = 45.0, time_stride: int = 5) -> pd.DataFrame:
    """Generate camera events e=(time, direction) that may cover multiple photo targets."""

    rows = []
```

```
    xy = traj[[X_COL, Y_COL]].to_numpy(float)
    speed = traj["speed"].to_numpy(float)
    acc = traj["acceleration"].to_numpy(float)
    times = traj[TIME_COL].to_numpy(float)
    target_xy = photo_targets[[X_COL, Y_COL]].to_numpy(float)
    target_ids = photo_targets["编号"].astype(str).tolist()
    half = fov_degree / 2.0
    for i in range(5, len(traj), time_stride):
        if np.any(speed[i - 5 : i + 1] > 1.5) or np.any(acc[i - 5 : i + 1] > 1.5):
            continue
        vec = target_xy - xy[i]
        dist_now = np.sqrt(np.sum(vec * vec, axis=1))
        visible_idx = np.where((dist_now >= 10.0) & (dist_now <= 40.0))[0]
        if len(visible_idx) == 0:
            continue
        angles = np.degrees(np.arctan2(vec[visible_idx, 1], vec[visible_idx, 0]))
        directions = sorted(set(round(float(a) / 5.0) * 5.0 for a in angles))
        for phi in directions:
            covered = []
            covered_angles = []
            covered_dist = []
            for idx, ang in zip(visible_idx, angles):
                if circular_angle_diff(phi, ang) <= half:
                    # Distance must hold through the preparation window for this target.
                    dwin = np.sqrt(np.sum((target_xy[idx] - xy[i - 5 : i + 1]) ** 2, axis=1))
                    if np.all((dwin >= 10.0) & (dwin <= 40.0)):
                        covered.append(target_ids[idx])
                        covered_angles.append(float(ang))
                        covered_dist.append(float(dist_now[idx]))
            if not covered:
                continue
            all_margins = []
            for tid in covered:
                pidx = target_ids.index(tid)
                dwin = np.sqrt(np.sum((target_xy[pidx] - xy[i - 5 : i + 1]) ** 2, axis=1))
                all_margins.append(_margin(dwin, speed[i - 5 : i + 1], acc[i - 5 : i + 1], 10.0, 40.0, 1.5, 1.5))
            rows.append(
                {
                    "event_id": f"P_{len(rows):05d}",
                    "event_type": "photo",
                    "start_time": float(times[i - 5]),
                    "execute_time": float(times[i]),
                    "camera_direction": float(phi),
                    "covered_targets": ",".join(covered),
                    "num_covered_targets": int(len(covered)),
                    "target_angles": ",".join(f"{a:.1f}" for a in covered_angles),
                    "speed": float(speed[i]),
                    "acceleration": float(acc[i]),
                    "min_distance": float(np.min(covered_dist)),
                    "max_distance": float(np.max(covered_dist)),
                    "margin": float(np.min(all_margins)),
                    "fov_degree": float(fov_degree),
                }
            )
    return pd.DataFrame(rows)
```

Listing 6: `src/task_events.py`（联合事件池与 MILP 求解核心段）

```
def _joint_pool(shoot_events: pd.DataFrame, photo_events: pd.DataFrame, min_margin: float = -np.inf) -> pd.DataFrame:
    shot_pool = shoot_events.copy()
    photo_pool = photo_events.copy()
    if not shot_pool.empty:
        shot_pool["covered_targets"] = shot_pool["target_id"]
        shot_pool["camera_direction"] = np.nan
        shot_pool["num_covered_targets"] = 1
        shot_pool["target_angles"] = ""
    pool = pd.concat([shot_pool, photo_pool], ignore_index=True, sort=False)
    if pool.empty:
        return pool
    pool = pool[pool["margin"].astype(float) >= min_margin].copy()
    pool = pool.sort_values(["execute_time", "event_type", "margin"], ascending=[True, True, False]).reset_index(drop=True)
    return pool


def _select_joint_plan_milp(
    shoot_events: pd.DataFrame,
```

```
    photo_events: pd.DataFrame,
    photo_targets: pd.DataFrame,
    max_shots_per_target: int = 5,
    min_margin: float = -np.inf,
    risk_weight: float = 0.20,
    time_limit: float = 60.0,
) -> tuple[pd.DataFrame, dict[str, object]]:
    pool = _joint_pool(shoot_events, photo_events, min_margin=min_margin)
    pruned_by_margin = int(len(shoot_events) + len(photo_events) - len(pool))
    if pool.empty:
        return pd.DataFrame(), {
            "solver": "scipy.milp",
            "status": "empty_candidate_pool",
            "objective_value": 0.0,
            "mip_gap": np.nan,
            "runtime_sec": 0.0,
            "n_variables": 0,
            "n_constraints": 0,
            "n_candidate_events": 0,
            "n_time_conflict_edges": 0,
            "n_photo_angle_conflict_edges": 0,
            "n_pruned_by_margin": pruned_by_margin,
            "used_fallback": False,
        }

    n_events = len(pool)
    shooting_target_ids = sorted(shoot_events["target_id"].astype(str).unique()) if not shoot_events.empty else []
    photo_target_ids = photo_targets["编号"].astype(str).tolist()
    y_index: dict[tuple[str, int], int] = {}
    idx = n_events
    for tid in shooting_target_ids:
        for m in range(1, max_shots_per_target + 1):
            y_index[(tid, m)] = idx
            idx += 1
    u_index = {}
    for tid in photo_target_ids:
        u_index[tid] = idx
        idx += 1
    n_vars = idx

    rows: list[np.ndarray] = []
    lb: list[float] = []
    ub: list[float] = []
    time_conflict_edges = 0
    photo_angle_conflict_edges = 0

    def add_constraint(coeffs: dict[int, float], lower: float, upper: float) -> None:
        row = np.zeros(n_vars)
        for pos, val in coeffs.items():
            row[pos] = val
        rows.append(row)
        lb.append(lower)
        ub.append(upper)

    # Time-window conflicts.
    for i in range(n_events):
        for j in range(i + 1, n_events):
            if _conflicts(float(pool.loc[i, "start_time"]), float(pool.loc[i, "execute_time"]), float(pool.loc[j, "start_time"]), float(pool.loc[j, "execute_time"])):
                add_constraint({i: 1.0, j: 1.0}, 0.0, 1.0)
                time_conflict_edges += 1

    # Shooting count equals selected marginal slots; y_m is ordered.
    for tid in shooting_target_ids:
        event_idxs = pool.index[(pool["event_type"] == "shooting") & (pool["target_id"].astype(str) == tid)].tolist()
        coeff = {i: 1.0 for i in event_idxs}
        for m in range(1, max_shots_per_target + 1):
            coeff[y_index[(tid, m)]] = coeff.get(y_index[(tid, m)], 0.0) - 1.0
        add_constraint(coeff, 0.0, 0.0)
        for m in range(2, max_shots_per_target + 1):
            add_constraint({y_index[(tid, m)]: 1.0, y_index[(tid, m - 1)]: -1.0}, -np.inf, 0.0)

    # Photo coverage variables cannot be set without at least one selected covering event.
    for tid in photo_target_ids:
        cover_idxs = []
        for i, ev in pool[pool["event_type"] == "photo"].iterrows():
            if tid in str(ev["covered_targets"]).split(","):
                cover_idxs.append(i)
```

```
        coeff = {u_index[tid]: 1.0}
        for i in cover_idxs:
            coeff[i] = coeff.get(i, 0.0) - 1.0
        add_constraint(coeff, -np.inf, 0.0)

    # Same-target photo angles must differ by at least 60 degrees.
    photo_part = pool[pool["event_type"] == "photo"]
    for tid in photo_target_ids:
        ev_angles = []
        for i, ev in photo_part.iterrows():
            targets = str(ev["covered_targets"]).split(",")
            angles = [float(a) for a in str(ev["target_angles"]).split(",") if a != ""]
            if tid in targets and len(angles) == len(targets):
                ev_angles.append((i, angles[targets.index(tid)]))
        for a_pos, (i, ai) in enumerate(ev_angles):
            for j, aj in ev_angles[a_pos + 1 :]:
                if circular_angle_diff(ai, aj) < 60.0:
                    add_constraint({i: 1.0, j: 1.0}, 0.0, 1.0)
                    photo_angle_conflict_edges += 1

    c = np.zeros(n_vars)
    # SciPy minimizes. Coefficients below maximize the stated weighted utility.
    m1, m2, m3, m4, m5 = 1000.0, 10.0, 2.0, 0.25, risk_weight
    for (tid, m), pos in y_index.items():
        c[pos] = -m1 * SHOT_HIT_PROB * (1.0 - SHOT_HIT_PROB) ** (m - 1)
    for pos in u_index.values():
        c[pos] = -m1
    for i, ev in pool.iterrows():
        margin = float(ev["margin"])
        if ev["event_type"] == "photo":
            covered_count = int(ev.get("num_covered_targets", 1))
            c[i] += -(m2 * covered_count + m3 * max(0, covered_count - 1))
        c[i] += -(m4 * margin) + m5 * _risk_penalty(margin)

    integrality = np.ones(n_vars)
    bounds = Bounds(np.zeros(n_vars), np.ones(n_vars))
    constraints = LinearConstraint(np.vstack(rows), np.asarray(lb), np.asarray(ub)) if rows else None
    result = milp(c=c, integrality=integrality, bounds=bounds, constraints=constraints, options={"time_limit": time_limit, "mip_rel_gap": 0.0})
    status = {
        "solver": "scipy.milp",
        "status": str(getattr(result, "message", "")) if getattr(result, "success", False) else f"failed:{getattr(result, 'message', '')}",
        "objective_value": float(-result.fun) if getattr(result, "fun", None) is not None else np.nan,
        "mip_gap": float(getattr(result, "mip_gap", np.nan)),
        "runtime_sec": float(getattr(result, "time", np.nan)),
        "n_variables": int(n_vars),
        "n_constraints": int(len(rows)),
        "n_candidate_events": int(n_events),
        "n_shooting_events": int((pool["event_type"] == "shooting").sum()),
        "n_photo_events": int((pool["event_type"] == "photo").sum()),
        "n_time_conflict_edges": int(time_conflict_edges),
        "n_photo_angle_conflict_edges": int(photo_angle_conflict_edges),
        "n_pruned_by_margin": int(pruned_by_margin),
        "weight_M1_target_completion": 1000.0,
        "weight_M2_photo_observation": 10.0,
        "weight_M3_multi_target_photo": 2.0,
        "weight_M4_margin": 0.25,
        "weight_M5_risk": float(risk_weight),
        "used_fallback": False,
    }
    if not getattr(result, "success", False) or result.x is None:
        fallback = select_joint_plan(shoot_events, photo_events, max_shots_per_target=max_shots_per_target)
        status["used_fallback"] = True
        return fallback, status

    selected_idx = np.where(result.x[:n_events] > 0.5)[0]
    out = pool.loc[selected_idx].copy().sort_values("execute_time").reset_index(drop=True)
    if not out.empty:
        shot_counts: dict[str, int] = {}
        shot_indices = []
        cum_probs = []
        gains = []
        for _, ev in out.iterrows():
            if ev["event_type"] == "shooting":
                tid = str(ev["target_id"])
                shot_counts[tid] = shot_counts.get(tid, 0) + 1
                m = shot_counts[tid]
                shot_indices.append(m)
                gains.append(SHOT_HIT_PROB * (1.0 - SHOT_HIT_PROB) ** (m - 1))
```

```
                cum_probs.append(1.0 - (1.0 - SHOT_HIT_PROB) ** m)
            else:
                shot_indices.append(np.nan)
                gains.append(np.nan)
                cum_probs.append(np.nan)
        out.insert(0, "seq", np.arange(1, len(out) + 1))
        out["shot_index_for_target"] = shot_indices
        out["marginal_hit_gain"] = gains
        out["cumulative_hit_probability"] = cum_probs
    return out, status
```

Listing 7: `src/task_events.py`（主方案与保守方案调度流程）

```
def run_event_task_optimization(traj: pd.DataFrame, targets: pd.DataFrame, out_dir: Path, fov_main: float = 45.0) -> dict[str, pd.DataFrame]:
    """Run pure shooting, pure photo, and joint event-level task plans."""

    tables = out_dir / "tables"
    figs = out_dir / "figures"
    tables.mkdir(parents=True, exist_ok=True)
    figs.mkdir(parents=True, exist_ok=True)
    shooting_targets, photo_targets = split_targets(targets)
    shoot_events = generate_shooting_events(traj, shooting_targets, stride=1)
    pure_shoot, shooting_summary = select_shooting_plan(shoot_events, shooting_targets)
    photo_events = generate_photo_events(traj, photo_targets, fov_degree=fov_main, time_stride=5)
    pure_photo, photo_summary = select_photo_plan(photo_events, photo_targets)
    joint, solver_status = _select_joint_plan_milp(shoot_events, photo_events, photo_targets, min_margin=-np.inf, risk_weight=0.20)
    conservative, conservative_status = _select_joint_plan_milp(shoot_events, photo_events, photo_targets, min_margin=0.03, risk_weight=3.00)
    robustness = robustness_table(joint)
    shooting_summary = summarize_shooting_targets(shoot_events, joint, shooting_targets)
    photo_summary = summarize_photo_targets(photo_events, joint, photo_targets)
    candidate_stats = candidate_generation_stats(traj, shooting_targets, photo_targets, shoot_events, photo_events, solver_status)
    tradeoff, tradeoff_plans = risk_tradeoff_curve(shoot_events, photo_events, photo_targets)

    fov_rows = []
    for fov in [30.0, 45.0, 60.0]:
        events = generate_photo_events(traj, photo_targets, fov_degree=fov, time_stride=5)
        selected, _summary = select_photo_plan(events, photo_targets)
        metrics = plan_metrics(f"pure_photo_fov_{int(fov)}", selected)
        fov_rows.append(
            {
                "fov_degree": fov,
                "selected_events": metrics["photo_events"],
                "covered_targets": metrics["photo_targets_covered"],
                "total_target_observations": metrics["photo_observations"],
                "multi_target_events": metrics["multi_target_photo_events"],
                "mean_margin": metrics["mean_margin"],
            }
        )

    comparison = pd.DataFrame(
        [
            plan_metrics("pure_shooting", pure_shoot, solver_status="shooting_greedy_slot"),
            plan_metrics("pure_photo", pure_photo, solver_status="photo_greedy_set_packing"),
            plan_metrics("joint", joint, solver_status="scipy.milp" if not solver_status["used_fallback"] else "greedy_interval_fallback"),
        ]
    )
    solver_df = pd.DataFrame([solver_status | {"plan_name": "joint"}, conservative_status | {"plan_name": "joint_conservative"}])
    conservative_df = conservative_compare(joint, conservative)
```

Listing 8: `scripts/reproduce_model_results_v4.py`

```
from __future__ import annotations

import os
import sys
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT / "src"))
OUT = ROOT / "outputs"
os.environ.setdefault("MPLCONFIGDIR", str(OUT / ".mplconfig"))

import pandas as pd

from bias_structure import run_attachment3_bias_structure
```

```python
from data_io import read_position_workbook, read_targets
from fill_result import fill_result_template_v2
from fusion import fuse_attachment
from kalman_bias import run_kalman_bias_attachment2
from kinematics import add_kinematics
from task_events import run_event_task_optimization


def ensure_dirs() -> None:
    for sub in ["tables", "figures", "trajectories", "logs"]:
        (OUT / sub).mkdir(parents=True, exist_ok=True)


def main() -> None:
    """Full v4 reproduction for teacher-feedback model iteration."""

    ensure_dirs()
    att2 = read_position_workbook(ROOT / "附件2.xlsx")
    att3 = read_position_workbook(ROOT / "附件3.xlsx")
    sheet1_2 = next(s for s in att2 if "方式1" in s)
    sheet2_2 = next(s for s in att2 if "方式2" in s)
    sheet1_3 = next(s for s in att3 if "方式1" in s)
    sheet2_3 = next(s for s in att3 if "方式2" in s)

    print("Running problem 2 Kalman-RTS bias model...")
    kalman_summary, kalman_sens, _kalman_traj = run_kalman_bias_attachment2(att2[sheet1_2], att2[sheet2_2], OUT)
    print(kalman_summary[["method", "Delta", "bx", "by", "rmse_after"]].to_string(index=False))

    print("Running problem 3 bias-structure identification...")
    bias_cmp, _coeffs, _residual_frame = run_attachment3_bias_structure(att3[sheet1_3], att3[sheet2_3], OUT)
    print(bias_cmp[["model_name", "cv_rmse", "improvement_vs_M1", "conclusion"]].to_string(index=False))

    print("Building attachment 3 trajectory for event-level task optimization...")
    _result3, fused3 = fuse_attachment(att3, estimate_bias=True, smooth_window=7)
    traj3 = add_kinematics(fused3, smooth_window=81)
    traj3.to_csv(OUT / "trajectories" / "fused_attachment3_v4_10hz.csv", index=False, encoding="utf-8-sig")
    targets = read_targets(ROOT / "附件4.xlsx")

    print("Running problem 4 event-level shooting/photo/joint optimization...")
    task_outputs = run_event_task_optimization(traj3, targets, OUT, fov_main=45.0)
    joint = task_outputs["joint_selected"]
    fill_result_template_v2(ROOT / "result.xlsx", OUT / "result_filled_v2.xlsx", joint)
    print(task_outputs["comparison"].to_string(index=False))

    notes = [
        "# v4 model iteration outputs",
        "",
        "- Problem 2 adds Kalman Filter / RTS smoother for random-noise separation and fixed source-2 bias estimation.",
        "- Problem 3 compares M0-M4 residual bias structures with blocked cross-validation.",
        "- Problem 4 replaces the old 9-row task cap with event-level pure shooting, pure photo, and joint plans.",
        "- Joint task solver_status is a deterministic greedy interval fallback; no 9-task capacity constraint is used.",
    ]
    (OUT / "logs" / "v4_iteration_summary.md").write_text("\n".join(notes), encoding="utf-8")


if __name__ == "__main__":
    main()
```

C AIGC 使用声明

本文写作和修改过程中使用了豆包、Kimi、DeepSeek、千问等人工智能工具进行辅助，主要用途包括文字润色、表述精简、结构检查和代码调试建议。论文中的模型假设、算法选择、数据处理、程序实现、数值结果、图表生成和结论判断均由作者完成并复核。所有 AIGC 输出均经过人工筛选、修改和校验，未直接作为未经核查的结论使用。

D 逐目标完整结果表

表 23 18 个射击目标逐目标结果

目标	可行事件	选择次数	命中概率	最小裕度	风险	未选原因
S01	52	0	0.0000	–	高	time_conflict
S02	27	0	0.0000	–	高	time_conflict
S03	28	1	0.8500	0.116	低	–
S04	28	0	0.0000	–	高	time_conflict
S05	28	0	0.0000	–	高	time_conflict
S06	23	1	0.8500	0.034	中	–
S07	0	0	0.0000	–	高	no_feasible_window
S08	0	0	0.0000	–	高	no_feasible_window
S09	0	0	0.0000	–	高	no_feasible_window
S10	252	5	0.9999	0.062	中	–
S11	226	5	0.9999	0.080	低	–
S12	202	5	0.9999	0.009	高	–
S13	4	1	0.8500	0.003	高	–
S14	37	1	0.8500	0.029	高	–
S15	37	1	0.8500	0.056	中	–
S16	37	1	0.8500	0.025	高	–
S17	0	0	0.0000	–	高	no_feasible_window
S18	0	0	0.0000	–	高	no_feasible_window

表 24 18 个拍照目标逐目标结果

目标	可行事件	选择次数	观测角/度	覆盖事件	未选原因
P01	16	1	-80.1	P_00035	–
P02	28	1	-61.2	P_00035	–
P03	9	1	-9.6	P_00012	–
P04	10	1	11.7	P_00012	–
P05	0	0	–	–	no_feasible_window
P06	0	0	–	–	no_feasible_window
P07	0	0	–	–	no_feasible_window
P08	0	0	–	–	no_feasible_window
P09	0	0	–	–	no_feasible_window
P10	5	1	-69.5	P_00020	–
P11	5	1	-37.5	P_00001	–
P12	0	0	–	–	no_feasible_window
P13	0	0	–	–	no_feasible_window
P14	0	0	–	–	no_feasible_window
P15	0	0	–	–	no_feasible_window
P16	0	0	–	–	no_feasible_window
P17	0	0	–	–	no_feasible_window
P18	0	0	–	–	no_feasible_window

E 最终任务表与填表说明

表 25 给出主方案 A 的完整事件序列。填写最终结果模板时，射击事件填写单个射击目标编号；拍照事件若覆盖多个目标，则在目标编号列写作如 P03,P04，并保持同一事件编号和同一执行时刻。模板红色说明文字不被修改。

表 25 主方案 A 最终任务事件表

序号	事件	类型	覆盖目标	准备开始/s	执行/s	裕度
1	S_00169	射击	S06	479.45	480.95	0.034
2	S_00106	射击	S03	481.55	483.05	0.116
3	P_00001	拍照	P11	493.05	493.55	0.011
4	P_00012	拍照	P03,P04	494.05	494.55	0.217
5	P_00020	拍照	P10	495.05	495.55	0.072
6	S_00665	射击	S12	506.75	508.25	0.009
7	S_00457	射击	S11	508.55	510.05	0.314
8	P_00035	拍照	P01,P02	510.55	511.05	0.184
9	S_00719	射击	S12	513.75	515.25	0.087
10	S_00509	射击	S11	515.35	516.85	0.278
11	S_00751	射击	S12	516.95	518.45	0.192
12	S_00526	射击	S11	522.15	523.65	0.125
13	S_00542	射击	S11	523.75	525.25	0.108
14	S_00306	射击	S10	525.35	526.85	0.147
15	S_00322	射击	S10	526.95	528.45	0.085
16	S_00333	射击	S10	531.45	532.95	0.121
17	S_00350	射击	S10	533.15	534.65	0.062
18	S_00366	射击	S10	534.75	536.25	0.108
19	S_00910	射击	S15	748.85	750.35	0.056
20	S_00889	射击	S14	750.45	751.95	0.029
21	S_00979	射击	S16	752.05	753.55	0.025
22	S_00866	射击	S13	762.05	763.55	0.003
23	S_00827	射击	S12	763.65	765.15	0.100
24	S_00644	射击	S11	765.55	767.05	0.080
25	S_00865	射击	S12	767.45	768.95	0.107

F MILP 集合、变量与派生量补充

表 26　问题四 MILP 集合、变量与派生量说明

符号	含义
$\mathcal{S}$	18 个射击目标集合，目标编号为 S01–S18。
$\mathcal{P}$	18 个拍照目标集合，目标编号为 P01–P18。
$\mathcal{E}^S$	射击候选事件集合；每个事件由射击目标、执行时刻和准备窗口确定。
$\mathcal{E}^P$	拍照候选事件集合；每个事件由执行时刻、相机方向和覆盖目标集合确定。
$\mathcal{E}$	联合候选事件集合，$\mathcal{E} = \mathcal{E}^S \cup \mathcal{E}^P$。
$\mathcal{E}_k^S$	属于射击目标 k 的全部候选射击事件。
C_e	拍照事件 e 覆盖的拍照目标集合；若 $\lvert C_e \rvert > 1$，表示多目标同拍。
$W_e = [s_e, t_e]$	事件 e 的准备窗口，射击为 $[t_e - 1.5, t_e]$，拍照为 $[t_e - 0.5, t_e]$。
$\mathcal{C}_T$	时间冲突边集合；若两个事件准备窗口重叠，则对应一条冲突边。
$\mathcal{C}_\theta$	拍照角度冲突边集合；若同一目标两次观测方向角差小于 60°，则对应一条冲突边。
q_e	统一事件选择变量，$q_e = 1$ 表示事件 e 被选入最终方案。
x_e	射击事件选择变量，仅对 $e \in \mathcal{E}^S$ 有效。
z_e	拍照事件选择变量，仅对 $e \in \mathcal{E}^P$ 有效。
n_k	射击目标 k 的实际射击次数。
$y_{k,m}$	目标 k 是否获得第 m 次射击的槽位变量，用于线性化递减命中收益。
u_p	拍照目标 p 是否至少被一个拍照事件覆盖。
m_e^{nom}	事件 e 在名义轨迹下的最小归一化裕度。
m_e^{worst}	事件 e 在全部残差扰动场景下的最坏裕度。
ρ_e	事件 e 的场景通过率，即扰动场景中仍满足约束的比例。
E_{shoot}	射击期望命中贡献，等于所有目标边际命中收益之和。
C_{photo}	至少被覆盖一次的拍照目标数量。
O_{photo}	所有被选拍照事件的目标观测总次数。
B_{multi}	多目标同拍奖励，衡量同一拍照事件覆盖多个目标的附加价值。
R_{risk}	风险惩罚项，按 $\pi(m_e^{\mathrm{nom}})$ 对低名义裕度事件分段计罚。

表 26 与正文中的完整约束一一对应。候选事件生成阶段确定集合 $\mathcal{E}^S, \mathcal{E}^P, \mathcal{C}_T, \mathcal{C}_\theta$，

鲁棒复核阶段确定 m_e^{worst} 与 ρ_e，MILP 求解阶段只对 $q_e, x_e, z_e, y_{k,m}, u_p$ 等 0–1 变量进行选择。

G 结果复现说明

为保证论文结果可复现，模型输出可分为轨迹结果、参数结果、任务结果和最终填表结果四类。轨迹结果对应问题一至问题三中的时间对齐、融合轨迹和运动学量；参数结果对应固定偏差估计、Q/R 灵敏度、偏差结构模型和鲁棒场景；任务结果对应候选事件、冲突边、MILP 求解状态和逐目标结果；最终填表结果用于提交任务安排。

填写最终结果模板时，拍照事件若覆盖多个目标，则在目标编号列写成如 P03,P04 的形式并保留同一事件编号和同一执行时刻。若模板必须拆行，仍保持相同执行时刻以表示同一拍照事件覆盖多个目标。

表 27 结果复现流程

步骤	复现内容	输出核验
1	读取附件 1–4 并统一字段、单位和时间列	时间单调、目标数量为射击 18 个和拍照 18 个。
2	对附件 1–3 分别进行时间偏差搜索和 10 Hz 插值	Δ、重叠区间、对齐前后 RMSE 与轨迹点数。
3	对附件 2 估计固定偏差并运行卡尔曼滤波、RTS 平滑	固定偏差 (b_x, b_y)、残差方差、Q/R 灵敏度和运动学抖动指标。
4	对附件 3 构造 M0–M4 偏差模型并做时间块 CV	训练 RMSE、时间块 CV-RMSE 和是否采用强校正的结论。
5	枚举射击、拍照候选事件并生成冲突边	候选数、剪枝数、时间冲突边和角度冲突边。
6	求解纯射击、纯拍照、联合 A 和保守 B 方案	求解器状态、目标值、gap、逐目标结果和 A/B 对比。
7	写入最终结果模板并生成论文图表	核对最终任务表、关键结果表和论文图表。

复现流程中，问题一到问题三的输出先确定轨迹状态和不确定性范围，问题四再基于该轨迹状态生成候选事件。这样的顺序使任务优化直接建立在时间对齐、偏差识别、运动学量计算和鲁棒场景复核的结果之上。